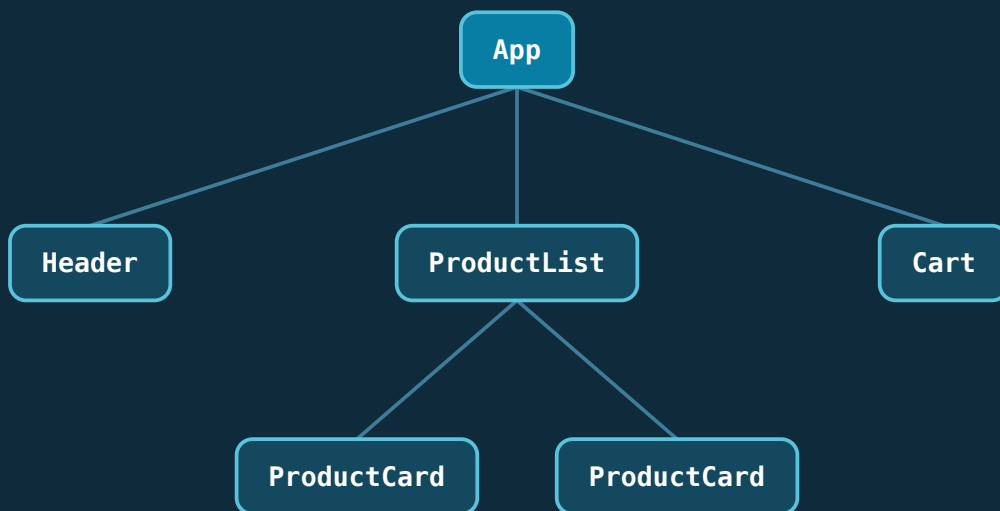




PRACTICAL LEARNING & PRACTICE WORKBOOK

React, Step by Step

From your first component to custom hooks,
for JavaScript developers who are new to React.

15 chapters · 45 practice tasks · 3 mini projects · 1 final challenge

Built with React 19 · Vite · React Router 8 · plain JavaScript
Edition: September 2026

Contents

Getting Started	4
How to Use This Workbook	4
Setting Up Your First React Project	6
 Part 1 • Foundations	 8
1. Component Basics	8
2. JSX	13
3. Props	18
4. Events & Event Handling	23
5. State (useState)	28
6. Props vs State	33
 Part 2 • Building Interfaces	 37
7. Conditional Rendering	37
8. Lists & .map()	41
9. Component Composition	46
10. Forms & Controlled Components	51
 Part 3 • Talking to the Outside World	 57
11. useEffect	57
12. API Calls / Fetch	63
 Part 4 • Scaling Up	 69
13. React Router	69
14. Context API	74
15. Custom Hooks	80
 Part 5 • Mini Projects	 86
Before You Build: Thinking in React	86
Mini Project 1: Todo App	87
Mini Project 2: Product Explorer	92
Mini Project 3: Team Dashboard	98
 Part 6 • Revision	 105

React Cheat Sheet	105
How the Concepts Fit Together	108
Final Practice Challenge: Recipe Finder	110
Final Challenge: Solution	112
Appendix	121
Quick Check Answers	121

How to Use This Workbook

This workbook teaches React by making you **build things**. Every chapter introduces one idea, shows it in a tiny example you can type in a few minutes, then asks you to change and extend that example yourself. By the end, you will have built three small applications and one final challenge without copying a tutorial.

You already know JavaScript: variables, functions, arrow functions, arrays and objects, `.map()`, `.filter()`, `.find()`, DOM basics, events, `fetch()` and JSON. This workbook builds on exactly that knowledge. It does **not** assume you have seen React before.

The 10-step pattern

Every one of the 15 topic chapters follows the same structure, so you always know where you are.

Step	What you get
What is it?	The idea in plain language, with no jargon
Why do we need it?	The real problem this feature solves
Syntax	The minimum code shape you need to remember
Small Example	A short program you can type by hand
Explain the Code	What the important lines do
What Happens?	What React is doing for you, at a conceptual level
Practice Task	2–3 exercises that change or extend the example
Solution	Simple answers, on the following pages
Common Mistakes	The errors almost every beginner makes
Quick Check	Short questions. Answers are in the appendix

The boxes you will see

REMEMBER

The one idea from a section you must not forget.

COMMON MISTAKE

A bug that beginners hit again and again, and how to avoid it.

BEHIND THE SCENES

What React is doing under the hood, explained without deep internals.

TIP

A practical habit that makes your life easier.

How to study with this book

1. **Type every example yourself.** Do not copy and paste. Typing builds the muscle memory that reading cannot.
2. **Break things on purpose.** Remove a **key**, forget an import, call a function too early. Read the error message. You will meet these errors again, and recognising them is a skill.
3. **Try the practice tasks before reading the solution.** Getting stuck for ten minutes teaches more than reading an answer in ten seconds.
4. **Answer the Quick Check out loud.** If you can explain it, you understand it.
5. **Keep the browser console open** (F12 or Ctrl+Shift+I / Cmd+Option+I). React prints helpful warnings there.

The learning path

The chapters build on each other in this order. Each arrow means "you need the thing above to understand the thing below".

Part 1 · Foundations	Components → JSX → Props → Events → State → Props vs State
Part 2 · Building UIs	Conditional Rendering → Lists → Composition → Forms
Part 3 · Outside World	useEffect → API Calls
Part 4 · Scaling Up	React Router → Context API → Custom Hooks
Part 5 · Mini Projects	Todo App → Product Explorer → Team Dashboard
Part 6 · Revision	Cheat Sheet → Concept Map → Final Challenge

Setting Up Your First React Project

What you need

- **Node.js**, a current LTS version. Check with `node -v` in a terminal. If the command is not found, install Node.js from nodejs.org.
- **A code editor**. VS Code is the most common choice.
- **A modern browser** such as Chrome, Edge or Firefox.
- Optional but very useful: the **React Developer Tools** browser extension, which lets you inspect components, props and state.

Create the project

We use **Vite**, a fast build tool that sets up React for you. Run these commands one at a time:

Terminal

```
npm create vite@latest react-workbook -- --template react
cd react-workbook
npm install
npm run dev
```

The last command prints a local address, usually `http://localhost:5173`. Open it in your browser. You should see the Vite starter page. Leave this command running: every time you save a file, the browser updates automatically.

What is inside the project

```
react-workbook/
├── index.html      ← the single HTML page; contains <div id="root">
├── package.json    ← project name, scripts and dependencies
└── src/
    ├── main.jsx    ← starts React and puts <App /> into #root
    ├── App.jsx      ← your main component (you will edit this a lot)
    ├── App.css
    └── index.css
```

Open `src/main.jsx`. It looks roughly like this:

`src/main.jsx`

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

This file finds the empty `<div id="root">` in `index.html` and tells React: "you are in charge of everything inside this div, and the first thing to show is the `App` component." You will rarely need to change this file.

StrictMode is a development helper. It does not show anything on screen; it runs extra checks and prints warnings. You will see one visible effect of it in Chapter 11.

Start with a clean page

Replace everything in `src/App.jsx` with this:

`src/App.jsx`

```
export default function App() {  
  return <h1>Hello React</h1>;  
}
```

Also delete the contents of `src/App.css` and `src/index.css` (keep the files, just empty them) so the starter styles do not confuse you. Save. The browser should now show only "Hello React".

TIP · How to run each exercise

Create a new file in `src/` for each example (for example `src/Counter.jsx`), then import it in `App.jsx` and render it. When you move to the next example, change what `App` renders. You can keep all your old files as a personal reference library.

NOTE · File extensions

Files that contain JSX (the HTML-like syntax) use the `.jsx` extension. Plain JavaScript files with no JSX, such as data files or custom hooks, can use `.js`.

CHAPTER 1

Component Basics

What is it?

A **component** is a reusable piece of user interface: a button, a card, a header, or even a whole page. In modern React, a component is simply **a JavaScript function that returns what should appear on the screen**.

Think of LEGO bricks. You build small bricks once, then snap them together into bigger things. A React application is a tree of components snapped together:

```
Page
├── Header
├── ProductCard
├── ProductCard
├── ProductCard
└── Footer
```

Why do we need it?

Without components, a web page is one long HTML file. If a product card appears 20 times, you copy its markup 20 times. When the design changes, you edit it in 20 places and probably miss one.

With components, you write `ProductCard` **once** and reuse it everywhere. Each piece lives in its own file, has one job, and is easy to understand on its own.

Syntax

```
function ComponentName() {
  return <h1>Some UI</h1>;
}

// Using it inside another component:
<ComponentName />
```

Three rules to remember:

- The function name **must start with a capital letter**: `Welcome`, not `welcome`.
- The function **returns JSX** (the HTML-like syntax you will study in Chapter 2).
- You use a component like an HTML tag: `<Welcome />`.

Small Example

The tree for this example is as small as it gets:

```
App
└── Welcome
```

src/App.jsx

```
function Welcome() {  
  return <h2>Welcome to the React Workbook!</h2>;  
}  
  
export default function App() {  
  return (  
    <div>  
      <h1>My First React App</h1>  
      <Welcome />  
    </div>  
  );  
}
```

As your app grows, each component gets its own file. Here is the same example split into two files:

src/Welcome.jsx

```
export default function Welcome() {  
  return <h2>Welcome to the React Workbook!</h2>;  
}
```

src/App.jsx

```
import Welcome from "../Welcome.jsx";  
  
export default function App() {  
  return (  
    <div>  
      <h1>My First React App</h1>  
      <Welcome />  
    </div>  
  );  
}
```

Explain the Code

Code	Meaning
<code>function Welcome()</code>	A normal JavaScript function. The capital W tells React it is a component.
<code>return <h2>...</h2></code>	Whatever the function returns is what this component shows.
<code>export default</code>	Makes the function available to other files.
<code>import Welcome from "../Welcome.jsx"</code>	Brings the component into <code>App.jsx</code> so it can be used there.
<code><Welcome /></code>	"Show the Welcome component here." The <code>/</code> closes the tag because nothing goes inside it.
<code>return (...)</code>	Parentheses let the JSX span several lines safely.

What Happens?

1. `main.jsx` tells React to show `<App />` inside `<div id="root">`.
2. React **calls** the `App` function. `App` returns JSX that contains `<Welcome />`.
3. React sees a tag starting with a capital letter, so it calls the `Welcome` function too.

4. React combines all the results into one description of the page and creates real DOM elements from it.

```
React calls App()  
  ↓  
App returns <div> with <h1> and <Welcome />  
  ↓  
React calls Welcome()  
  ↓  
Welcome returns <h2>  
  ↓  
React creates the real DOM: <div><h1>...</h1><h2>...</h2></div>
```

REMEMBER

A component is just a function, but **React calls it for you**. You never write `Welcome()` yourself; you write `<Welcome />` and React decides when to run it.

Practice Task

1. Create a `Profile` component that shows a person's name in an `<h3>` and their job title in a `<p>`. Render it inside `App`, below `Welcome`.
2. Create a `ProductCard` component that shows a product name, a price and a "Buy" button (the button does not need to do anything yet). Render it **three times** inside `App`.
3. Move `Profile` and `ProductCard` into their own files (`Profile.jsx` and `ProductCard.jsx`) and import them into `App.jsx`.

Solution

src/Profile.jsx

```
export default function Profile() {  
  return (  
    <div>  
      <h3>Asha Kumar</h3>  
      <p>Frontend Developer</p>  
    </div>  
  );  
}
```

src/ProductCard.jsx

```
export default function ProductCard() {  
  return (  
    <div className="product-card">  
      <h3>Wireless Mouse</h3>  
      <p>$19.99</p>  
      <button>Buy</button>  
    </div>  
  );  
}
```

src/App.jsx

```
import Welcome from './Welcome.jsx';
import Profile from './Profile.jsx';
import ProductCard from './ProductCard.jsx';

export default function App() {
  return (
    <div>
      <h1>My First React App</h1>
      <Welcome />
      <Profile />
      <ProductCard />
      <ProductCard />
      <ProductCard />
    </div>
  );
}
```

NOTE

All three cards show the same mouse. A component that can only ever show one thing is not very useful. Chapter 3 (Props) fixes this.

Common Mistakes

COMMON MISTAKE · Lowercase component names

✗ Wrong: React treats `<welcome>` as an unknown HTML tag

```
function welcome() { return <h2>Hi</h2>; }
<welcome />
```

✓ Right

```
function Welcome() { return <h2>Hi</h2>; }
<Welcome />
```

COMMON MISTAKE · Forgetting to return

A function with curly braces `{ }` must use `return`. Without it, the function returns `undefined` and nothing appears.

✗ Wrong: nothing is returned

```
function Welcome() {
  <h2>Hi</h2>;
}
```

COMMON MISTAKE · Import problems

If you see an error like "does not provide an export named default" or "Failed to resolve import", check three things: the file has `export default`, the path starts with `./`, and the file name matches exactly, including capital letters.

COMMON MISTAKE · Calling a component like a function

Write `<Welcome />`, not `{Welcome()}`. Calling it directly may seem to work now, but it breaks React features such as state (Chapter 5).

Quick Check

1. What is a React component, in one sentence?
2. Why must component names start with a capital letter?
3. What does `export default` do?
4. Who calls your component function: you, or React?

CHAPTER 2

JSX

What is it?

JSX is the HTML-like syntax you write inside JavaScript files. It looks like HTML, but it is really JavaScript underneath. That means you can mix in variables, calculations and function calls wherever you need them.

Why do we need it?

Building UI with plain DOM code is slow to write and hard to read:

Plain JavaScript DOM

```
const h1 = document.createElement("h1");
h1.className = "title";
h1.textContent = "Hello, " + name;
document.body.appendChild(h1);
```

The same thing in JSX

```
<h1 className="title">Hello, {name}</h1>
```

JSX lets you **describe the result** you want. React does the DOM work for you. This style is called **declarative**: you say *what* the UI should look like, not *how* to build it step by step.

Syntax

Four rules cover almost everything:

1. **Curly braces { }** mean "run JavaScript here".
2. **Return one parent element.** Use a Fragment `<> ... </>` if you do not want an extra `<div>`.
3. **Close every tag**, including `<img />`, `<br />` and `<input />`.
4. **Attributes use camelCase.** `class` becomes `className`, `for` becomes `htmlFor`, `onclick` becomes `onClick`.

```
const name = "Asha";
const imageUrl = "https://i.pravatar.cc/100";

<p>Hello, {name}!</p>
<p>2 + 3 = {2 + 3}</p>
<p>{name.toUpperCase()}</p>
<img src={imageUrl} alt="Avatar" />
```

Small Example

src/App.jsx

```
export default function App() {
  const user = { firstName: "Asha", lastName: "Kumar", birthYear: 1998 };
  const currentYear = new Date().getFullYear();
  const skills = ["HTML", "CSS", "JavaScript"];

  return (
    <>
      <h1 className="title">
        {user.firstName} {user.lastName}
      </h1>
      <p>Age: {currentYear - user.birthYear}</p>
      <p>Skills: {skills.join(", ")}</p>
      <p style={{ color: "green", fontWeight: "bold" }}>Available for work</p>
    </>
  );
}
```

Explain the Code

- `<> ... </>` is a **Fragment**. It groups several elements without adding an extra element to the page.
- `{user.firstName} {user.lastName}`: each pair of braces inserts a value. The space between them is ordinary text.
- `{currentYear - user.birthYear}`: any **expression** works inside braces, including maths and method calls.
- `{skills.join(", ")}` turns the array into the text "HTML, CSS, JavaScript".
- `style={{ ... }}`: the outer braces mean "JavaScript here"; the inner braces are a normal object. CSS property names become camelCase: `font-weight` becomes `fontWeight`.
- `className="title"` sets the CSS class, exactly like `class` in HTML.

JSX vs HTML

HTML	JSX	Why
<code>class="box"</code>	<code>className="box"</code>	<code>class</code> is a reserved word in JavaScript
<code>for="email"</code>	<code>htmlFor="email"</code>	<code>for</code> is a reserved word in JavaScript
<code>
</code> and <code></code>	<code>
</code> and <code></code>	every tag must be closed
<code>style="color: red"</code>	<code>style={{ color: "red" }}</code>	<code>style</code> is a JavaScript object
<code>onclick="save()"</code>	<code>onClick={save}</code>	camelCase, and you pass a function
<code><!-- comment --></code>	<code>{/* comment */}</code>	comments are JavaScript comments

NOTE • Expressions, not statements

Braces accept **expressions**, which are pieces of code that produce a value: `{price * 2}`, `{isAdmin ? "Admin" : "User"}`, `{getName()}`. They do **not** accept **statements** such as `if`, `for` or `const`. Chapter 7 shows how to write conditions and Chapter 8 shows how to write loops in JSX.

What Happens?

Browsers do not understand JSX. While you develop, Vite converts every JSX tag into an ordinary JavaScript function call. That call creates a small object that **describes** a piece of UI:

What your JSX becomes (simplified)

```
// You write:
<h1 className="title">Hello</h1>

// Vite turns it into roughly:
jsx("h1", { className: "title", children: "Hello" });
```

React reads these description objects and creates or updates the real DOM to match them. This also explains rule 2: a function can only return **one** value, so a component must return one root element.

Practice Task

1. Create `const userName = "Ravi"`. Show "Hello, Ravi!" in an `<h1>`, and underneath it show "Your name has 4 letters", calculating the number from the string.
2. Given `const price = 250`, `const quantity = 3` and `const taxRate = 0.18`, display the subtotal, the tax and the total. Show the total with two decimal places using `.toFixed(2)`.
3. Build a `ProfileCard` component from an object `{ name, city, role, avatar }`. Show the avatar in an `<img>` with a width of 80 pixels (use the `style` prop), the name in an `<h2>`, and "role · city" in a `<p>`. Give the outer `<div>` the class `profile-card`.

Solution

Exercises 1 and 2

```
export default function App() {
  const userName = "Ravi";
  const price = 250;
  const quantity = 3;
  const taxRate = 0.18;
  const subtotal = price * quantity;
  const tax = subtotal * taxRate;

  return (
    <>
      <h1>Hello, {userName}!</h1>
      <p>Your name has {userName.length} letters</p>

      <p>Subtotal: {subtotal}</p>
      <p>Tax: {tax}</p>
      <p>Total: {(subtotal + tax).toFixed(2)}</p>
    </>
  );
}
```

Exercise 3: src/ProfileCard.jsx

```
export default function ProfileCard() {
  const person = {
    name: "Meena Iyer",
    city: "Pune",
    role: "UX Designer",
    avatar: "https://i.pravatar.cc/100?img=5",
  };

  return (
    <div className="profile-card">
      <img src={person.avatar} alt={person.name} style={{ width: "80px" }} />
      <h2>{person.name}</h2>
      <p>
        {person.role} · {person.city}
      </p>
    </div>
  );
}
```

TIP

Calculate values in normal JavaScript **above** the **return**, then insert them with braces. Your JSX stays short and readable.

Common Mistakes

COMMON MISTAKE · Returning two elements without a wrapper

✗ Wrong: "Adjacent JSX elements must be wrapped in an enclosing tag"

```
return (
  <h1>Title</h1>
  <p>Text</p>
);
```

✓ Right: wrap them in a Fragment

```
return (
  <>
    <h1>Title</h1>
    <p>Text</p>
  </>
);
```

COMMON MISTAKE · Writing style as a string

`style="color: red"` causes an error. Use an object: `style={{ color: "red" }}`.

COMMON MISTAKE · Rendering a whole object

`<p>{user}</p>` throws "Objects are not valid as a React child". Render a property instead: `<p>{user.name}</p>`.

COMMON MISTAKE · Unclosed tags and class instead of className

`` is a syntax error in JSX; write ``. Using `class` still works but prints a warning in the console, so use `className`.

Quick Check

1. What do curly braces mean in JSX?
2. Why do we write `className` instead of `class`?
3. What is a Fragment, and when would you use one?
4. Will `{if (isAdmin) { ... }}` work inside JSX? Why or why not?
5. What happens if you try to render an object like `{user}`?

CHAPTER 3

Props

What is it?

Props (short for "properties") are how a parent component passes data to a child component. They work exactly like **function arguments**, and you write them like **HTML attributes** on your own components.

Why do we need it?

At the end of Chapter 1, all three `ProductCards` showed the same mouse. Props let **one component display different data** each time it is used.

```
App
└─ UserCard ← receives name="Asha", role="Designer"
```

Syntax

```
// The parent passes props like attributes:
<UserCard name="Asha" age={26} />

// The child receives all of them as ONE object:
function UserCard(props) {
  return <h3>{props.name}</h3>;
}

// Most developers destructure that object instead:
function UserCard({ name, age }) {
  return <h3>{name} ({age})</h3>;
}
```

Small Example

src/UserCard.jsx

```
export default function UserCard({ name, role, age }) {
  return (
    <div className="user-card">
      <h3>{name}</h3>
      <p>
        {role}, {age} years old
      </p>
    </div>
  );
}
```

src/App.jsx

```
import UserCard from "./UserCard.jsx";

export default function App() {
  return (
    <div>
      <UserCard name="Asha" role="Designer" age={26} />
      <UserCard name="Ravi" role="Developer" age={31} />
      <UserCard name="Meena" role="Tester" age={24} />
    </div>
  );
}
```

One component, three different results.

Explain the Code

- `name="Asha"`: the parent sets a prop called `name`. **Strings** can use quotes.
- `age={26}`: **numbers need braces**. `age="26"` would be the string "26", not the number 26.
- `({ name, role, age })`: destructuring pulls the three values out of the props object.
- Each `<UserCard ... />` is a separate use of the same function with different arguments.

Passing different kinds of data

Anything that is not a plain string goes inside braces:

```
<ProductCard
  title="Laptop"                // string
  price={999}                  // number
  inStock={true}               // boolean
  tags=[["electronics", "office"]] // array
  seller={{ name: "TechMart", rating: 4.5 }} // object (note the double braces)
/>
```

```
function ProductCard({ title, price, inStock, tags, seller }) {
  return (
    <div>
      <h3>{title}</h3>
      <p>Price: ${price}</p>
      <p>{inStock ? "In stock" : "Sold out"}</p>
      <p>Tags: {tags.join(", ")}</p>
      <p>
        Sold by {seller.name} (rating {seller.rating})
      </p>
    </div>
  );
}
```

NOTE • The dollar sign

In `<p>Price: ${price}</p>` the `$` is ordinary text shown on screen, followed by `{price}`. It is **not** a template literal.

Default values

If a prop might be missing, give it a default right in the destructuring:

```
function UserCard({ name, role = "Guest" }) {
  return <p>{name} is a {role}</p>;
}
```

```
<UserCard name="Arun" /> // shows "Arun is a Guest"
```

What Happens?

When React sees `<UserCard name="Asha" role="Designer" age={26} />`, it collects every attribute into one object and calls your function with it:

```
UserCard({ name: "Asha", role: "Designer", age: 26 })
```

Data always flows **one way: from parent to child**. When the parent's data changes, React calls the child again with the new props, and the screen updates.

```
App — props —> UserCard
(owns the data)   (displays the data)
```

REMEMBER

Props are read-only. A child must never change its own props. If some data needs to change over time, it becomes **state**, which you will learn in Chapter 5.

Practice Task

1. **UserCard:** accept `name`, `email` and `isAdmin` props. Show the text "Admin" next to the name only when `isAdmin` is `true` (use a ternary). Render one admin and one normal user.
2. **ProductCard:** accept `title`, `price` and a `discount` prop (a percentage, default 0). Show the original price and the final price after discount.
3. **StudentCard:** accept a **single** prop called `student`, an object like `{ name, grade, subjects: ["Maths", "Physics"] }`. Show the name, the grade, how many subjects there are, and the subjects joined by commas.

Solution

Exercise 1

```
function UserCard({ name, email, isAdmin }) {
  return (
    <div className="user-card">
      <h3>
        {name} {isAdmin ? "(Admin)" : ""}
      </h3>
      <p>{email}</p>
    </div>
  );
}

export default function App() {
  return (
    <>
      <UserCard name="Asha" email="asha@example.com" isAdmin={true} />
      <UserCard name="Ravi" email="ravi@example.com" isAdmin={false} />
    </>
  );
}
```

Exercise 2

```
function ProductCard({ title, price, discount = 0 }) {
  const finalPrice = price - (price * discount) / 100;

  return (
    <div>
      <h3>{title}</h3>
      <p>Original: ${price}</p>
      <p>
        Now: ${finalPrice} ({discount}% off)
      </p>
    </div>
  );
}

// Usage:
// <ProductCard title="Headphones" price={80} discount={25} />
// <ProductCard title="Charger" price={15} />
```

Exercise 3

```
function StudentCard({ student }) {
  return (
    <div>
      <h3>{student.name}</h3>
      <p>Grade: {student.grade}</p>
      <p>
        {student.subjects.length} subjects: {student.subjects.join(", ")}
      </p>
    </div>
  );
}

// Usage:
// <StudentCard
//   student={{ name: "Kiran", grade: "A", subjects: ["Maths", "Physics"] }}
// />
```

Common Mistakes

COMMON MISTAKE · Numbers passed as strings

✗ Wrong: price is the string "100", so price + 10 is "10010"

```
<ProductCard price="100" />
```

✓ Right

```
<ProductCard price={100} />
```

COMMON MISTAKE · Forgetting the destructuring braces

function Card(title) gives you the **whole props object** in a variable named title. Write function Card({ title }).

COMMON MISTAKE · Changing props inside the child

props.name = "New name" is not allowed. Props belong to the parent. To change data, the parent must own it as state and pass down a function to change it (Chapters 4 and 6).

COMMON MISTAKE · Name mismatch

If the parent writes `userName="Asha"` but the child reads `{ name }`, then `name` is `undefined` and nothing shows. The names must match exactly.

Quick Check

1. In which direction do props flow?
2. What is the difference between `age="26"` and `age={26}`?
3. Can a child component change its props?
4. How do you give a prop a default value?
5. What will `<Card />` show if `Card` renders `{title}` but no `title` is passed?

CHAPTER 4

Events & Event Handling

What is it?

Events are things the user does: clicking a button, typing in an input, submitting a form. In React you attach a **handler function** directly in JSX using camelCase props such as `onClick`, `onChange` and `onSubmit`.

Why do we need it?

An app that cannot respond to the user is just a picture. In plain JavaScript you find the element first and then attach a listener. In React you attach the handler right where the element is written, so you can see the behaviour and the markup together.

Plain JavaScript

```
const button = document.querySelector("#save");
button.addEventListener("click", () => alert("Saved"));
```

React

```
<button onClick={() => alert("Saved")}>Save</button>
```

Syntax

```
function handleClick() {
  alert("Clicked");
}

<button onClick={handleClick}>Click</button>           // pass the function
<input onChange={(e) => console.log(e.target.value)} /> // inline arrow function
<form onSubmit={handleSubmit}>...</form>
```

Event prop	Fires when	Typical element
<code>onClick</code>	the element is clicked	button, div, link
<code>onChange</code>	the value of an input changes (every keystroke)	input, select, textarea
<code>onSubmit</code>	a form is submitted (Enter key or submit button)	form
<code>onMouseEnter</code> / <code>onMouseLeave</code>	the pointer moves over / away	any element
<code>onKeyDown</code>	a key is pressed	input

Small Example

src/EventDemo.jsx

```
export default function EventDemo() {
  function handleClick() {
    alert("Button clicked!");
  }

  function handleChange(event) {
    console.log("You typed:", event.target.value);
  }

  function handleSubmit(event) {
    event.preventDefault();
    alert("Form submitted!");
  }

  return (
    <div>
      <button onClick={handleClick}>Click me</button>

      <input placeholder="Type here" onChange={handleChange} />

      <form onSubmit={handleSubmit}>
        <input placeholder="Your email" />
        <button type="submit">Subscribe</button>
      </form>
    </div>
  );
}
```

Passing values to a handler

Wrap the call in an arrow function. The arrow function runs on click, and **it** calls your function with the value you want:

```
function greet(name) {
  alert(`Hello, ${name}!`);
}

<button onClick={() => greet("Asha")}>Greet Asha</button>
<button onClick={() => greet("Ravi")}>Greet Ravi</button>
```

Explain the Code

- `onClick={handleClick}` gives React the function **itself**. React will call it later, when the click happens.
- `handleChange(event)`: React passes an **event object** to every handler. `event.target` is the element that fired the event, so `event.target.value` is the input's current text.
- `event.preventDefault()` stops the browser's default behaviour. For a form, the default is to reload the page, which would wipe your app.
- `() => greet("Asha")` is a small function that runs only when clicked.

Handlers as props: how a child talks to its parent

Props only flow down. So how does a child tell its parent that something happened? The parent **passes a function down** as a prop, and the child calls it:

```
function LikeButton({ onLike }) {
  return <button onClick={onLike}>Like</button>;
}
```

```
}  
  
export default function App() {  
  function handleLike() {  
    console.log("Someone liked the post");  
  }  
  
  return <LikeButton onLike={handleLike} />;  
}
```

By convention, the **prop** is named `onSomething` and the **function** that handles it is named `handleSomething`.

What Happens?

React sets up the real browser listeners for you. When the user clicks, React finds the handler you attached to that element and calls it with an event object. React's event object behaves the same in every browser, so you do not have to worry about browser differences.

REMEMBER

`onClick={doThing}` **passes** the function, and React calls it on click. `onClick={doThing()}` **calls** the function immediately while the screen is being drawn. That is almost always a bug.

NOTE • Console logs do not change the screen

Everything in this chapter only logs or shows alerts. To actually **change what is displayed** when the user clicks, you need **state**. That is the next chapter.

Practice Task

1. Add two buttons: "Say Hi" logs "Hi!" to the console, and "What time is it?" shows an alert with the current time (`new Date().toLocaleTimeString()`).
2. Add an input that logs how many characters have been typed every time the value changes.
3. Create a form with an input named `username`. On submit, prevent the page reload and show an alert "Welcome, (name)!". Read the value with `event.target.elements.username.value`.

Solution

Exercises 1 to 3

```
export default function Practice() {
  function handleTime() {
    alert(new Date().toLocaleTimeString());
  }

  function handleLength(e) {
    console.log("Characters:", e.target.value.length);
  }

  function handleSubmit(e) {
    e.preventDefault();
    const name = e.target.elements.username.value;
    alert(`Welcome, ${name}!`);
  }

  return (
    <div>
      <button onClick={() => console.log("Hi!")}>Say Hi</button>
      <button onClick={handleTime}>What time is it?</button>

      <input placeholder="Type something" onChange={handleLength} />

      <form onSubmit={handleSubmit}>
        <input name="username" placeholder="Your name" />
        <button type="submit">Enter</button>
      </form>
    </div>
  );
}
```

TIP

Reading form values from `event.target.elements` works, but React developers usually keep input values in state instead. You will learn that better approach, called **controlled components**, in Chapter 10.

Common Mistakes

COMMON MISTAKE · Calling the handler instead of passing it

✗ Wrong: runs during render, not on click

```
<button onClick={handleClick()}>Save</button>
```

✓ Right

```
<button onClick={handleClick}>Save</button>
<button onClick={() => handleClick(42)}>Save item 42</button>
```

COMMON MISTAKE · Lowercase or string handlers

`onclick={...}` (lowercase) and `onClick="handleClick()"` (a string) do not work in React. Use camelCase and pass a function in braces.

COMMON MISTAKE · Forgetting preventDefault on forms

Without `event.preventDefault()`, submitting a form reloads the page. Your console is cleared and any data on the screen is lost, which makes it look as if "nothing happened".

COMMON MISTAKE · Changing a normal variable and expecting the screen to update

`let count = 0; count++` inside a click handler changes the variable but the screen stays the same. This is exactly the problem state solves.

Quick Check

1. What is the difference between `onClick={save}` and `onClick={save()}`?
2. How do you pass an argument such as an ID to a click handler?
3. What does `event.preventDefault()` do in a submit handler?
4. How can a child component notify its parent that a button was clicked?

CHAPTER 5

State (useState)

What is it?

State is a component's **memory**. It is data that can change over time and that affects what is shown on the screen: a counter value, whether a menu is open, the items in a cart.

Why do we need it?

Why not just use a normal variable? Try it:

✗ This does not work

```
function Counter() {  
  let count = 0;  
  
  function increase() {  
    count = count + 1;  
    console.log(count); // prints 1, 2, 3... in the console  
  }  
  
  return <button onClick={increase}>Count: {count}</button>; // always shows 0  
}
```

There are two problems here:

1. **React does not know the variable changed**, so it never redraws the component.
2. Even if React did redraw it, the function would run again from the top, and `let count = 0` would **reset** the value.

`useState` solves both problems. It keeps the value safe **between** renders, and it gives you a setter function that tells React "this changed, please redraw".

Syntax

```
import { useState } from "react";  
  
const [value, setValue] = useState(initialValue);  
  
const [count, setCount] = useState(0);  
      |         |           |  
      current  function that  starting value  
      value    updates it     (used only on the first render)
```

`useState` is a **hook**. Hooks are special React functions whose names start with `use`. You will write your own hooks in Chapter 15.

Small Example

src/Counter.jsx

```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(count - 1)}>-1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Explain the Code

- `import { useState } from "react"` brings the hook into the file.
- `useState(0)` creates a piece of state that starts at 0.
- `const [count, setCount]` uses array destructuring. The names are your choice; the pattern `thing / setThing` is the convention.
- `setCount(count + 1)` asks React to store a new value **and re-render** the component.
- On every render, `count` holds the latest stored value.

What Happens?

1st render	<code>useState(0) → count = 0</code>	screen shows "Count: 0"
click	<code>setCount(1) → React schedules a re-render</code>	
2nd render	React calls <code>Counter()</code> again	
	<code>useState(0) → count = 1</code> (the 0 is now ignored)	
	React compares the new JSX with the old one	
	and updates only the text that changed → "Count: 1"	

A **re-render** means React calls your component function again to get fresh JSX. It then updates only the parts of the real DOM that are different. It does not rebuild the whole page.

Updating based on the previous value

Look at this surprising example:

```
function addThree() {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);
}
// Clicking once adds only 1, not 3!
```

During one render, `count` is a fixed snapshot. If `count` is 0, all three lines say `setCount(0 + 1)`. The fix is to pass an **updater function**. React gives it the latest value each time:

```
function addThree() {
  setCount((prev) => prev + 1);
  setCount((prev) => prev + 1);
  setCount((prev) => prev + 1);
}
// Now it adds 3
```

REMEMBER

If the new value **depends on the old value**, use the updater form: `setCount(prev => prev + 1)`.

State can hold any value

```
const [isVisible, setIsVisible] = useState(false);
setIsVisible((prev) => !prev);           // toggle a boolean

const [name, setName] = useState("");
setName("Asha");                        // string

const [user, setUser] = useState({ name: "Asha", age: 26 });
setUser({ ...user, age: 27 });           // copy the object, change one field

const [items, setItems] = useState(["Pen"]);
setItems([...items, "Book"]);            // add to an array
setItems(items.filter((i) => i !== "Pen")); // remove from an array
```

COMMON MISTAKE · Never change state directly

`user.age = 27` or `items.push("Book")` changes the existing object or array, but React does not notice, so the screen does not update. Always create a **new** object or array with the spread operator `...`, `.map()` or `.filter()`. We say state is **immutable**: you replace it; you never modify it in place.

Practice Task

1. **Show/hide text:** a button toggles a paragraph on and off. The button label should say "Show" or "Hide" depending on the state.
2. **Like button:** show "Like (12)". Clicking it once changes the label to "Liked (13)"; clicking again goes back to "Like (12)". Use one boolean state and one number state.
3. **Quantity selector:** show `-`, a number, and `+` buttons. The quantity must stay between 1 and 10. Below it, show the total price for a unit price of 120.

Solution

Exercise 1

```
import { useState } from "react";

export default function ToggleText() {
  const [isVisible, setIsVisible] = useState(false);

  return (
    <div>
      <button onClick={() => setIsVisible((v) => !v)}>
        {isVisible ? "Hide" : "Show"}
      </button>
      {isVisible ? <p>Surprise! This text can hide.</p> : null}
    </div>
  );
}
```

Exercise 2

```
import { useState } from "react";

export default function LikeButton() {
  const [liked, setLiked] = useState(false);
  const [likes, setLikes] = useState(12);

  function handleClick() {
    setLikes((n) => (liked ? n - 1 : n + 1));
    setLiked((l) => !l);
  }

  return (
    <button onClick={handleClick}>
      {liked ? "Liked" : "Like"} ({likes})
    </button>
  );
}
```

Exercise 3

```
import { useState } from "react";

const UNIT_PRICE = 120;

export default function QuantitySelector() {
  const [qty, setQty] = useState(1);

  return (
    <div>
      <button onClick={() => setQty((q) => Math.max(1, q - 1))}>-</button>
      <span> {qty} </span>
      <button onClick={() => setQty((q) => Math.min(10, q + 1))}>+</button>
      <p>Total: {qty * UNIT_PRICE}</p>
    </div>
  );
}
```

Notice that the total is **not** stored in state. It is calculated from `qty` on every render. If a value can be calculated from existing state, do not create extra state for it.

Common Mistakes

COMMON MISTAKE · Expecting the value to change immediately

```
setCount(count + 1);
console.log(count); // still the OLD value
```

The new value appears on the **next** render. Log it in the JSX, or calculate the new value in a variable first.

COMMON MISTAKE · Calling the setter during render

x Wrong: "Too many re-renders"

```
function Counter() {  
  const [count, setCount] = useState(0);  
  setCount(count + 1); // runs every render, which causes another render...  
  return <p>{count}</p>;  
}
```

Only call setters inside event handlers (or effects, Chapter 11).

COMMON MISTAKE · Calling hooks inside conditions or loops

Hooks must be called at the **top level** of the component, in the same order every render. Never write `if (x) { useState(...) }`. This is one of the official **Rules of Hooks**.

Quick Check

1. Why does a normal `let` variable not work for a counter?
2. What two things does `useState` return?
3. When should you use `setCount(prev => prev + 1)` instead of `setCount(count + 1)`?
4. Why is `items.push("x")` wrong for state? What should you write instead?
5. After `setCount(5)`, does `count` equal 5 on the very next line?

CHAPTER 6

Props vs State

What is it?

Both props and state are **data used to render a component**. The difference is **who owns the data** and **who is allowed to change it**.

Why do we need it?

Choosing the wrong one causes real bugs: data that does not update, the same data stored in two places, or a component that cannot tell another component about a change. Knowing the difference is the key to designing React apps.

Syntax

Question	Props	State
Who owns the data?	The parent component	The component itself
Where does it come from?	Passed in like attributes: <code><Card title="Hi" /></code>	Created inside with <code>useState()</code>
Can the component change it?	No, props are read-only	Yes, with its setter function
What happens when it changes?	The child re-renders with the new props	The component re-renders
Analogy	Arguments passed to a function	The function's own memory
Example	A product's name and price	Whether a dropdown is open

```
function Child({ title }) {  
  const [open, setOpen] = useState(false);  
  // ...  
}
```

// title is a PROP
// open is STATE

Small Example

This example uses both. Watch how one component's **state** becomes another component's **props**.

src/App.jsx

```
import { useState } from "react";

function CartBadge({ count }) {
  return <p>Cart: {count} items</p>;
}

function ProductCard({ name, price, onAdd }) {
  const [quantity, setQuantity] = useState(1);

  return (
    <div className="product-card">
      <h3>{name}</h3>
      <p>
        ${price} × {quantity} = ${price * quantity}
      </p>
      <button onClick={() => setQuantity((q) => q + 1)}>+</button>
      <button onClick={() => onAdd(quantity)}>Add to cart</button>
    </div>
  );
}

export default function App() {
  const [cartCount, setCartCount] = useState(0);

  function handleAdd(qty) {
    setCartCount((c) => c + qty);
  }

  return (
    <div>
      <CartBadge count={cartCount} />
      <ProductCard name="Notebook" price={3} onAdd={handleAdd} />
      <ProductCard name="Pen" price={1} onAdd={handleAdd} />
    </div>
  );
}
```

Explain the Code

- `cartCount` is **state in App**. App owns it, and only App can change it.
- `CartBadge` receives it as the **prop count**. `CartBadge` only displays it.
- `name` and `price` are **props** of `ProductCard`: the parent decides them.
- `quantity` is **state inside ProductCard**. Each card has its own separate quantity.
- `onAdd` is a **function prop**. When clicked, the card calls it, and App updates its own state.

What Happens?

App	state: cartCount	
├─ CartBadge	prop: count={cartCount}	
├─ ProductCard	props: name, price, onAdd	state: quantity
└─ ProductCard	props: name, price, onAdd	state: quantity

Click "Add to cart" on Notebook (quantity 2):

ProductCard calls `onAdd(2)`

→ App runs `setCartCount(c => c + 2)`

→ App re-renders

→ `CartBadge` receives `count={2}` and shows "Cart: 2 items"

This pattern is called **lifting state up**: when two components need the same changing data, you store the state in their **closest common parent** and pass it down as props.

Decision guide

Does this data change over time?

- └ No → use a prop (or a plain constant)
- └ Yes → Does THIS component change it?
 - └ Yes → state, in this component
 - └ No → prop (the parent owns the state)

Can it be calculated from other props or state?

- └ Yes → neither! Calculate it during render. (e.g. `total = price * quantity`)

REMEMBER

State lives in **one** place. Everyone else receives it as props. If two siblings need the same data, lift the state up to their common parent.

Practice Task

1. Add a `-` button to `ProductCard` that never lets the quantity go below 1.
2. Add a "Clear cart" button in `App` that resets the cart count to 0.
3. Add a third product, and make `CartBadge` say "Cart is empty" when the count is 0, "1 item" for one, and "N items" otherwise.

Solution

Changes for exercises 1 to 3

```
// 1. Inside ProductCard, next to the + button:
<button onClick={() => setQuantity((q) => Math.max(1, q - 1))}>-</button>

// 2. Inside App's return:
<button onClick={() => setCartCount(0)}>Clear cart</button>

// 3. A smarter CartBadge, and a third product:
function CartBadge({ count }) {
  if (count === 0) return <p>Cart is empty</p>;
  return <p>Cart: {count} {count === 1 ? "item" : "items"}</p>;
}

<ProductCard name="Eraser" price={0.5} onAdd={handleAdd} />
```

Common Mistakes

COMMON MISTAKE · Copying props into state

✗ Usually wrong

```
function Greeting({ name }) {
  const [displayName, setDisplayName] = useState(name);
  return <h1>{displayName}</h1>;
}
```

The initial value is used only on the first render. If the parent later passes a new `name`, the screen will not update. Just use the prop directly: `<h1>{name}</h1>`.

COMMON MISTAKE · Storing values you can calculate

Keeping `total` in state next to `price` and `quantity` means you must remember to update three things. Calculate `const total = price * quantity`; instead.

COMMON MISTAKE · State in the wrong component

If a sibling cannot "see" some data, the state is too low in the tree. Move it up to the common parent and pass it down.

Quick Check

1. Who owns props, and who owns state?
2. Can a component change the props it receives?
3. What does "lifting state up" mean?
4. In the example, why does each `ProductCard` have its own quantity?
5. Should `total = price × quantity` be props, state, or neither?

CHAPTER 7

Conditional Rendering

What is it?

Conditional rendering means showing different UI depending on a condition: a "Log out" button only for logged-in users, a spinner while data loads, a friendly message when a list is empty.

Why do we need it?

Real interfaces change all the time based on data. Without conditions you would need a separate page for every situation. With them, one component can handle every case: loading, error, empty and success.

Syntax

You cannot write `if` inside JSX braces, so React developers use these four patterns:

Pattern	Use it when	Example
<code>if</code> + early <code>return</code>	the whole output is different	<code>if (!user) return <Login />;</code>
Ternary <code>? :</code>	you choose between two pieces of UI	<code>{isOpen ? "On" : "Off"}</code>
<code>&&</code>	you show something or nothing	<code>{error && <p>{error}</p>}</code>
<code>return null</code>	the component should show nothing at all	<code>if (!show) return null;</code>

```
// 1. if + early return
if (isLoading) return <p>Loading...</p>;

// 2. ternary inside JSX
<h2>{isLoggedIn ? "Welcome back!" : "Please log in"}</h2>

// 3. && inside JSX
{hasError && <p className="error">Something went wrong</p>}

// 4. render nothing
if (!isOpen) return null;
```

Small Example

A login/logout panel:

src/AuthPanel.jsx

```
import { useState } from "react";

export default function AuthPanel() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const notifications = 3;

  return (
    <div>
      {isLoggedIn ? <h2>Welcome back, Asha!</h2> : <h2>Please log in</h2>}

      {isLoggedIn && notifications > 0 && (
        <p>You have {notifications} new notifications</p>
      )}

      <button onClick={() => setIsLoggedIn((prev) => !prev)}>
        {isLoggedIn ? "Log out" : "Log in"}
      </button>
    </div>
  );
}
```

Conditional components: loading and empty states

A very common shape is a component that checks several situations **in order** and returns early:

```
function ProductStatus({ loading, products }) {
  if (loading) return <p>Loading products...</p>;
  if (products.length === 0) return <p>No products found.</p>;
  return <p>{products.length} products available</p>;
}
```

Explain the Code

- `{isLoggedIn ? <h2>...</h2> : <h2>...</h2>}` picks one of two headings.
- `{isLoggedIn && notifications > 0 && (...)}`: if **every** condition is true, the JSX at the end is shown. If any is false, nothing is shown.
- The button label uses a ternary too: conditions work anywhere an expression is allowed.
- In `ProductStatus`, the order matters: first "is it loading?", then "is it empty?", and only then the normal display.

What Happens?

React re-evaluates every condition on each render. Values like `false`, `null` and `undefined` render **nothing**, which is why `{false && <X />}` shows nothing.

When a condition switches from one branch to another, React removes the old elements from the page and creates the new ones.

COMMON MISTAKE · The number zero with &&

✗ Shows a "0" on the screen when the list is empty

```
{items.length && <ItemList items={items} />}
```

0 is falsy, so && returns 0, and React **does** display numbers.

✓ Always compare to get a real true/false

```
{items.length > 0 && <ItemList items={items} />}
```

Practice Task

1. **Show/hide message:** a button toggles a message using && (not a ternary).
2. **Loading indicator:** create state `isLoading` starting as `true`. Show "Loading..." while it is true, and a "Finish loading" button that sets it to `false`, which reveals the content "Data is ready!".
3. **Empty product list:** keep `products` as an array in state, starting empty. Show "No products yet" when it is empty. A button adds "Product 1", "Product 2" and so on. When the list is not empty, show the count and the name of the last product added.

Solution

Exercise 1

```
import { useState } from "react";

export default function Message() {
  const [show, setShow] = useState(false);
  return (
    <div>
      <button onClick={() => setShow((s) => !s)}>Toggle message</button>
      {show && <p>Hello! You toggled me on.</p>}
    </div>
  );
}
```

Exercise 2

```
import { useState } from "react";

export default function Loader() {
  const [isLoading, setIsLoading] = useState(true);

  if (isLoading) {
    return (
      <div>
        <p>Loading...</p>
        <button onClick={() => setIsLoading(false)}>Finish loading</button>
      </div>
    );
  }

  return <p>Data is ready!</p>;
}
```

Exercise 3

```
import { useState } from "react";

export default function ProductBox() {
  const [products, setProducts] = useState([]);

  function addProduct() {
    setProducts((prev) => [...prev, `Product ${prev.length + 1}`]);
  }

  return (
    <div>
      <button onClick={addProduct}>Add product</button>
      {products.length === 0 ? (
        <p>No products yet</p>
      ) : (
        <p>
          {products.length} products. Latest: {products[products.length - 1]}
        </p>
      )}
    </div>
  );
}
```

Common Mistakes

COMMON MISTAKE · Trying to write if inside JSX

`{if (x) { ... }}` is a syntax error. Use a ternary or `&&`, or put the `if` above the `return`.

COMMON MISTAKE · Deeply nested ternaries

`a ? (b ? <X/> : <Y/>) : (c ? <Z/> : null)` is hard to read. Use early returns, or move each case into a small component.

COMMON MISTAKE · Forgetting the empty and loading states

Beginners often only write the "happy path". Always ask: what does the user see while waiting, when something fails, and when there is no data?

Quick Check

1. Name three ways to render something conditionally.
2. When would you use `&&` instead of a ternary?
3. Why can `{count && <p>...</p>}` show a "0"?
4. What does a component show if it returns `null`?

CHAPTER 8

Lists & .map()

What is it?

Rendering a list means turning an **array of data** into an **array of JSX elements**, usually with JavaScript's `.map()` method, which you already know.

Why do we need it?

Data from an API almost always arrives as an array: products, users, messages. You cannot hard-code 50 `<ProductCard />` tags, especially when you do not know in advance how many items there will be.

Syntax

```
const fruits = ["Apple", "Mango", "Banana"];

<ul>
  {fruits.map((fruit) => (
    <li key={fruit}>{fruit}</li>
  ))}
</ul>
```

`.map()` returns a new array of `<li>` elements, and React knows how to display an array of elements.

Small Example

```
ProductList
└─ ProductCard  (one for each item in the array)
```

src/ProductList.jsx

```
const products = [
  { id: 1, name: "Keyboard", price: 45, inStock: true },
  { id: 2, name: "Mouse", price: 20, inStock: false },
  { id: 3, name: "Monitor", price: 180, inStock: true },
];

function ProductCard({ product }) {
  return (
    <li className="product-card">
      <strong>{product.name}</strong> - ${product.price}
      {!product.inStock && <span> (out of stock)</span>}
    </li>
  );
}

export default function ProductList() {
  return (
    <ul>
      {products.map((product) => (
        <ProductCard key={product.id} product={product} />
      ))}
    </ul>
  );
}
```

Explain the Code

- `products` is an array of **objects**. Each object is one product.
- `products.map((product) => (...))` creates one `<ProductCard>` per product.
- `product={product}` passes the whole object as a prop. Inside, we read `product.name` and `product.price`.
- `key={product.id}` gives each element a **unique, stable identity**. The key goes on the element returned directly from `map`.

Why keys are required

When a list changes (an item is added, removed or reordered), React compares the old list with the new one. The key tells React **which item is which**. Without good keys, React may reuse the wrong element. For example, text typed into an input can "jump" to a different row after a delete.

Key choice	Verdict
<code>key={item.id}</code> from your data or database	✓ Best choice
<code>key={item.email}</code> or another value that is unique and never changes	✓ Fine
<code>key={index}</code> (the position in the array)	⚠ Only if the list never reorders, filters or deletes
<code>key={Math.random()}</code>	✗ A new key every render, so React rebuilds everything

NOTE

Keys only need to be unique **among siblings** in the same list, not across the whole app. React does **not** pass `key` to your component as a prop, so `product.key` or `props.key` will be `undefined`.

Filtering and mapping together

```
{products
  .filter((p) => p.inStock)
  .map((p) => (
    <ProductCard key={p.id} product={p} />
  ))}
```

What Happens?

```
[ {id: 1, ...}, {id: 2, ...}, {id: 3, ...} ]
  ↓ .map()
[ <ProductCard key=1 />, <ProductCard key=2 />, <ProductCard key=3 /> ]
  ↓ React renders the array
<li>Keyboard</li> <li>Mouse</li> <li>Monitor</li>
```

Later the array changes to `[ {id: 1}, {id: 3} ]`:
 React compares keys → key 2 is gone → it removes only that `<li>`.

Practice Task

1. **Student list:** create an array of four students `{ id, name, marks }`. Render each name and mark, plus "Pass" if marks are 40 or more, otherwise "Fail".
2. **Product list filter:** add a checkbox "Show in-stock only" to the example. Store it in state and filter the list when it is checked.
3. **Todo list:** keep todos in state as `{ id, text }`. An "Add sample task" button adds `{ id: Date.now(), text: "Task N" }`. Each todo has a "Delete" button that removes it.

Solution

Exercise 1

```
const students = [
  { id: 1, name: "Arjun", marks: 78 },
  { id: 2, name: "Divya", marks: 35 },
  { id: 3, name: "Farhan", marks: 52 },
  { id: 4, name: "Lakshmi", marks: 91 },
];

export default function StudentList() {
  return (
    <ul>
      {students.map((s) => (
        <li key={s.id}>
          {s.name}: {s.marks} - {s.marks >= 40 ? "Pass" : "Fail"}
        </li>
      ))}
    </ul>
  );
}
```

Exercise 2 (replace the ProductList function)

```
import { useState } from "react";

export default function ProductList() {
  const [inStockOnly, setInStockOnly] = useState(false);

  const visible = inStockOnly ? products.filter((p) => p.inStock) : products;

  return (
    <div>
      <label>
        <input
          type="checkbox"
          checked={inStockOnly}
          onChange={(e) => setInStockOnly(e.target.checked)}
        />
        Show in-stock only
      </label>
      <ul>
        {visible.map((p) => (
          <ProductCard key={p.id} product={p} />
        ))}
      </ul>
    </div>
  );
}
```

Exercise 3

```
import { useState } from "react";

export default function TodoList() {
  const [todos, setTodos] = useState([]);

  function addTodo() {
    setTodos((prev) => [
      ...prev,
      { id: Date.now(), text: `Task ${prev.length + 1}` },
    ]);
  }

  function deleteTodo(id) {
    setTodos((prev) => prev.filter((t) => t.id !== id));
  }

  return (
    <div>
      <button onClick={addTodo}>Add sample task</button>
      {todos.length === 0 && <p>No tasks yet.</p>}
      <ul>
        {todos.map((t) => (
          <li key={t.id}>
            {t.text} <button onClick={() => deleteTodo(t.id)}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Common Mistakes

COMMON MISTAKE · Using curly braces in map without return

✗ Wrong: the arrow function returns undefined, so nothing shows

```
{items.map((item) => {
  <li key={item.id}>{item.name}</li>;
})}
```

✓ Right: use () for an implicit return, or write return

```
{items.map((item) => (
  <li key={item.id}>{item.name}</li>
))}
```

COMMON MISTAKE · Missing keys, or keys on the wrong element

The console warns "Each child in a list should have a unique key prop". Put the key on the **outermost** element returned by `map`, which is `<ProductCard key=... />`, not the `<li>` inside `ProductCard`.

COMMON MISTAKE · Using push to add items

`todos.push(newTodo)` mutates state and the screen will not update. Use `setTodos([...todos, newTodo])`.

Quick Check

1. What does `.map()` return when used in JSX?
2. What is the `key` prop used for?
3. Why is the array index a risky key for a list you can delete from?
4. Where exactly should the `key` be placed?
5. How do you show only some of the items?

CHAPTER 9

Component Composition

What is it?

Composition means building complex UIs by **combining** small, simple components, including passing whole pieces of UI **into** a component through a special prop called **children**.

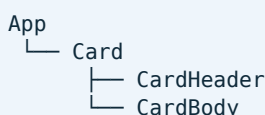
Why do we need it?

Many parts of an app share the same structure: a card, a modal, a page layout. You want to write that structure **once** and fill it with different content each time. You also want each component to do **one job**, so that it stays small and easy to change.

Syntax

```
function Card({ children }) {  
  return <div className="card">{children}</div>;  
}  
  
// Whatever you put between the tags becomes `children`:  
<Card>  
  <h3>Anything</h3>  
  <p>can go inside</p>  
</Card>
```

Small Example



src/Card.jsx

```
export function Card({ children }) {  
  return <div className="card">{children}</div>;  
}  
  
export function CardHeader({ title, subtitle }) {  
  return (  
    <div className="card-header">  
      <h3>{title}</h3>  
      {subtitle && <small>{subtitle}</small>}  
    </div>  
  );  
}  
  
export function CardBody({ children }) {  
  return <div className="card-body">{children}</div>;  
}
```

src/App.jsx

```
import { Card, CardHeader, CardBody } from "./Card.jsx";

export default function App() {
  return (
    <div>
      <Card>
        <CardHeader title="Profile" subtitle="Personal details" />
        <CardBody>
          <p>Name: Asha Kumar</p>
          <p>City: Pune</p>
        </CardBody>
      </Card>

      <Card>
        <CardHeader title="Settings" />
        <CardBody>
          <button>Change password</button>
        </CardBody>
      </Card>
    </div>
  );
}
```

src/index.css

```
.card { border: 1px solid #ddd; border-radius: 8px; margin: 12px 0; }
.card-header { padding: 12px 16px; border-bottom: 1px solid #eee; }
.card-body { padding: 12px 16px; }
```

Explain the Code

- `Card` and `CardBody` do not know what they contain. They just render `{children}` inside their wrapper.
- `CardHeader` uses normal props (`title`, `subtitle`) because its content is always the same shape.
- The two cards have completely different content but share one structure and one style.
- `export function Card` is a **named export**. Named exports are imported with braces: `import { Card, CardHeader } from "./Card.jsx"`. A file can have many named exports but only one `export default`.

What Happens?

`children` is just a prop. When React sees `<CardBody><p>Name</p></CardBody>`, it calls `CardBody({ children: <p>Name</p> })`. Your component decides **where** to place that content.

<pre><Card> <CardHeader ... /> <CardBody> <p>...</p> </CardBody> </Card></pre>	<pre>Card({ children: [CardHeader, CardBody] }) └─ renders <div class="card"> + children CardBody({ children: [<p>, <p>] }) └─ renders <div class="card-body"> + children</pre>
--	---

Thinking in components

Before you write code, sketch the screen and draw boxes around each part:



**TIP · When to create a new component**

Extract a component when (a) you are copying the same JSX a second time, (b) a part of the screen has its own clear job, or (c) a file grows beyond about 150 lines.

Practice Task

1. Create a reusable **Button** component that takes **children** and a **variant** prop ("primary" or "danger", default "primary") and gives the button the classes **btn** plus **btn-primary** or **btn-danger**.
2. Create a **Section** component with a **title** prop that renders an **<h2>** followed by its **children**. Use it twice with different content.
3. Build a **ProductCard** out of **Card**, **CardHeader**, **CardBody** and your **Button**, showing a product's name, price and a "Buy" button.

Solution

Exercises 1 to 3

```
import { Card, CardHeader, CardBody } from "./Card.jsx";

function Button({ children, variant = "primary", onClick }) {
  return (
    <button className={`btn btn-${variant}`} onClick={onClick}>
      {children}
    </button>
  );
}

function Section({ title, children }) {
  return (
    <section>
      <h2>{title}</h2>
      {children}
    </section>
  );
}

function ProductCard({ name, price }) {
  return (
    <Card>
      <CardHeader title={name} subtitle={`$${price}`} />
      <CardBody>
        <Button onClick={() => alert(`Bought ${name}`)}>Buy</Button>
      </CardBody>
    </Card>
  );
}

export default function App() {
  return (
    <>
      <Section title="Featured">
        <ProductCard name="Desk Lamp" price={25} />
      </Section>
      <Section title="Danger zone">
        <Button variant="danger">Delete account</Button>
      </Section>
    </>
  );
}
```

In the `subtitle` of `ProductCard`, the template literal starts with a plain-text `$` followed by `${price}`, which inserts the value and produces "\$25".

Common Mistakes

COMMON MISTAKE · Forgetting to render children

If a wrapper component does not include `{children}` in its JSX, everything you put between its tags silently disappears.

COMMON MISTAKE · Mixing up default and named imports

`export default function Card` is imported as `import Card from ...` (no braces). `export function Card` is imported as `import { Card } from ...` (with braces). Getting this wrong gives "does not provide an export named...".

COMMON MISTAKE · Giant components

A single 400-line component that does everything is hard to read and reuse. Split it into pieces that each do one job.

Quick Check

1. What is the `children` prop?
2. When should you use `children` instead of a normal prop?
3. How do you import a named export?
4. Give two signs that part of a component should become its own component.

CHAPTER 10

Forms & Controlled Components

What is it?

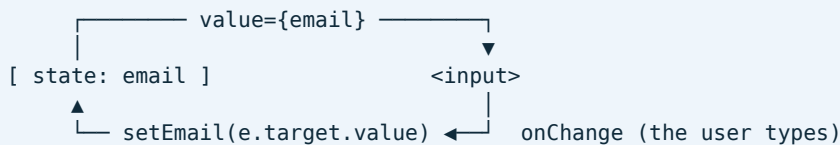
A **controlled component** is an input whose value is stored in React **state**. The input displays whatever the state says, and every keystroke updates the state. React becomes the **single source of truth** for the form.

Why do we need it?

When the value lives in state, you always know it. That makes it easy to validate while the user types, to disable the submit button until the form is valid, to reset the form, or to send the data to an API.

Syntax

```
const [email, setEmail] = useState("");  
  
<input value={email} onChange={(e) => setEmail(e.target.value)} />
```



Small Example

LoginForm

src/LoginForm.jsx

```
import { useState } from "react";

export default function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(event) {
    event.preventDefault();

    if (!email.includes("@")) {
      setError("Please enter a valid email.");
      return;
    }
    if (password.length < 6) {
      setError("Password must be at least 6 characters.");
      return;
    }

    setError("");
    alert(`Logged in as ${email}`);
    setEmail("");
    setPassword("");
  }

  return (
    <form onSubmit={handleSubmit}>
      <h2>Login</h2>

      <label htmlFor="email">Email</label>
      <input
        id="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />

      <label htmlFor="password">Password</label>
      <input
        id="password"
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />

      {error && <p style={{ color: "red" }}>{error}</p>}

      <button type="submit" disabled={!email || !password}>
        Log in
      </button>
    </form>
  );
}
```

Explain the Code

- Each input has **both** `value` (state → input) and `onChange` (input → state).
- `onSubmit` is on the `<form>`, not on the button, so it also works when the user presses Enter.
- `event.preventDefault()` stops the page reload.
- Validation is plain JavaScript. If something is wrong, set an error message and `return` early.
- `disabled={!email || !password}` keeps the button disabled until both fields have text.
- Resetting the form is just setting the state back to `""`.

NOTE

We used a plain text input for the email on purpose, so you can see our own validation. With `type="email"`, the browser also blocks invalid emails before your handler runs.

Many fields, one state object

For bigger forms, store all fields in one object and use **one** change handler. Each input needs a `name` attribute that matches a key in the object:

```
const [form, setForm] = useState({ name: "", email: "", city: "Pune", agree: false });

function handleChange(e) {
  const { name, value, type, checked } = e.target;
  setForm((prev) => ({
    ...prev,
    [name]: type === "checkbox" ? checked : value,
  }));
}


```

`[name]: value` is a **computed property name**. If `name` is `"email"`, it becomes `email: value`.

Input type	Controlled by	Read from the event
text, email, password, number	<code>value</code>	<code>e.target.value</code>
textarea	<code>value</code>	<code>e.target.value</code>
select	<code>value</code> on the <code><select></code>	<code>e.target.value</code>
checkbox	<code>checked</code>	<code>e.target.checked</code>
radio	<code>checked={form.size === "M"}</code>	<code>e.target.value</code>

What Happens?

Every keystroke runs `onChange`, which calls the setter. React re-renders, and the input displays the new state value. It feels instant, but the input is only ever showing what React tells it to. That is exactly why a `value` without an `onChange` gives you an input you cannot type into.

Practice Task

- Login form:** add a "Show password" checkbox that switches the password input's `type` between `"password"` and `"text"`.
- Registration form:** create a form with name, email, password, confirm password, a city `<select>` and a "I accept the terms" checkbox, using **one** state object and **one** change handler.
- Simple validation:** on submit, collect all errors (name required, email contains "@", password at least 6 characters, passwords match, terms accepted) and show them as a list. If there are no errors, show "Registered!" and reset the form.

Solution

Exercise 1 (inside LoginForm)

```
const [showPassword, setShowPassword] = useState(false);

<input
  id="password"
  type={showPassword ? "text" : "password"}
  value={password}
  onChange={(e) => setPassword(e.target.value)}
/>
<label>
  <input
    type="checkbox"
    checked={showPassword}
    onChange={(e) => setShowPassword(e.target.checked)}
  />
  Show password
</label>
```

Exercises 2 and 3: src/RegisterForm.jsx

```

import { useState } from "react";

const emptyForm = {
  name: "", email: "", password: "", confirm: "", city: "Pune", agree: false,
};

export default function RegisterForm() {
  const [form, setForm] = useState(emptyForm);
  const [errors, setErrors] = useState([]);
  const [success, setSuccess] = useState(false);

  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setForm((prev) => ({ ...prev, [name]: type === "checkbox" ? checked : value }));
  }

  function validate() {
    const list = [];
    if (!form.name.trim()) list.push("Name is required.");
    if (!form.email.includes("@")) list.push("Email must contain @.");
    if (form.password.length < 6) list.push("Password needs 6+ characters.");
    if (form.password !== form.confirm) list.push("Passwords do not match.");
    if (!form.agree) list.push("You must accept the terms.");
    return list;
  }

  function handleSubmit(e) {
    e.preventDefault();
    const found = validate();
    setErrors(found);
    setSuccess(found.length === 0);
    if (found.length === 0) setForm(emptyForm);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" placeholder="Name"
        value={form.name} onChange={handleChange} />
      <input name="email" placeholder="Email"
        value={form.email} onChange={handleChange} />
      <input name="password" type="password" placeholder="Password"
        value={form.password} onChange={handleChange} />
      <input name="confirm" type="password" placeholder="Confirm password"
        value={form.confirm} onChange={handleChange} />

      <select name="city" value={form.city} onChange={handleChange}>
        <option>Pune</option>
        <option>Delhi</option>
        <option>Mumbai</option>
      </select>

      <label>
        <input name="agree" type="checkbox"
          checked={form.agree} onChange={handleChange} />
        I accept the terms
      </label>

      {errors.length > 0 && (
        <ul style={{ color: "red" }}>
          {errors.map((err) => <li key={err}>{err}</li>)}
        </ul>
      )}

      {success && <p style={{ color: "green" }}>Registered!</p>}

      <button type="submit">Register</button>
    </form>
  );
}

```

```
    );  
  }
```

Common Mistakes

COMMON MISTAKE · value without onChange

The input becomes read-only and React warns: "You provided a value prop to a form field without an onChange handler". Always add `onChange`, or use `defaultValue` if you truly do not want to control it.

COMMON MISTAKE · Using value for checkboxes

A checkbox's on/off state is `checked`, and you read it from `e.target.checked`. Using `value` and `e.target.value` gives you the string "on" every time.

COMMON MISTAKE · Forgetting the name attribute with a shared handler

With one `handleChange`, `e.target.name` decides which field to update. If an input has no `name` (or a name that does not match a key), its value goes to the wrong place.

COMMON MISTAKE · Starting with undefined

`useState()` with no initial value makes the input switch from uncontrolled to controlled, and React warns about it. Start text fields with `""`.

Quick Check

1. What makes an input "controlled"?
2. Why do we put `onSubmit` on the form instead of `onClick` on the button?
3. How do you control a checkbox?
4. What does `[name]: value` do inside an object?
5. How do you reset a controlled form?

CHAPTER 11

useEffect

What is it?

`useEffect` lets you run code **after React has updated the screen**. You use it for work that reaches **outside** React: fetching data, starting a timer, changing the page title, saving to `localStorage`, or listening to window events. This kind of work is called a **side effect**.

Why do we need it?

A component's main job is to **calculate UI**: given these props and this state, return this JSX. React may call your component many times, so that calculation must be quick and must not "do" things to the outside world.

If you started a timer or a fetch directly in the component body, it would run on **every render**, creating dozens of timers or requests, and often an infinite loop. `useEffect` gives side effects a safe place to live, and lets you control **when** they run.

REMEMBER

Rendering should be **pure**: the same props and state should always produce the same JSX, with no side effects. Side effects go in event handlers (for things the user does) or in `useEffect` (for things that should happen because of a render).

Syntax

```
import { useEffect } from "react";

useEffect(() => {
  // side-effect code runs after the render

  return () => {
    // optional cleanup: undo the effect
  };
}, [dependencies]);
```

The three forms: the most important table in this chapter

The second argument, the **dependency array**, decides when the effect runs.

You write	It runs	Typical use
<code>useEffect(() => { })</code>	After every render	Rarely needed
<code>useEffect(() => { }, [])</code>	Once , after the first render	Load initial data, start a subscription
<code>useEffect(() => { }, [count])</code>	After the first render, and whenever <code>count</code> changes	Keep something in sync with a value

Try all three at once and watch the console:

src/EffectDemo.jsx

```
import { useState, useEffect } from "react";

export default function EffectDemo() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState("");

  useEffect(() => {
    console.log("A: after EVERY render");
  });

  useEffect(() => {
    console.log("B: ONCE, after the first render");
  }, []);

  useEffect(() => {
    console.log("C: count changed to", count);
  }, [count]);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Count: {count}</button>
      <input value={name} onChange={(e) => setName(e.target.value)} />
    </div>
  );
}
```

What you do	A runs?	B runs?	C runs?
The page loads	✓	✓	✓
You click the button	✓	✗	✓
You type in the input	✓	✗	✗

NOTE · Why do I see every log twice at first?

In development, `<StrictMode>` (from `main.jsx`) mounts your component, removes it, and mounts it again, so effects run twice on the first load. This is on purpose: it helps you find effects that are missing a cleanup. In production, they run once.

Small Example

A timer that also updates the browser tab title:

src/Timer.jsx

```
import { useState, useEffect } from "react";

export default function Timer() {
  const [seconds, setSeconds] = useState(0);
  const [running, setRunning] = useState(false);

  useEffect(() => {
    document.title = `Timer: ${seconds}s`;
  }, [seconds]);

  useEffect(() => {
    if (!running) return;

    const id = setInterval(() => {
      setSeconds((s) => s + 1);
    }, 1000);

    return () => clearInterval(id);
  }, [running]);

  return (
    <div>
      <h2>{seconds} seconds</h2>
      <button onClick={() => setRunning(!running)}>
        {running ? "Pause" : "Start"}
      </button>
      <button onClick={() => setSeconds(0)}>Reset</button>
    </div>
  );
}
```

Explain the Code

- The **first effect** keeps the tab title in sync with `seconds`. It depends on `seconds`, so it runs every time the number changes.
- The **second effect** depends on `running`. When `running` becomes `true`, it starts an interval.
- `return () => clearInterval(id)` is the **cleanup**. React runs it before the effect runs again and when the component is removed from the page.
- `setSeconds((s) => s + 1)` uses the updater form. The interval callback was created once, so a plain `seconds + 1` would keep using the old value.

The cleanup concept

Think of an effect as turning on a tap, and the cleanup as turning it off.

running: false → true	effect runs	→ interval started
running: true → false	cleanup runs first	→ interval cleared
	then the effect runs	→ "if (!running) return" (does nothing)
component removed	cleanup runs	→ interval cleared

Without the cleanup, pausing would not stop the old interval, and each "Start" would add another one, so the timer would speed up.

What Happens?

1. React calls your component and updates the DOM.
2. The browser paints the screen, so the user sees the result.
3. React checks each effect's dependency array. If any value is **different** from the last render (or it is the first render), it runs that effect's cleanup (if there is one) and then the effect.

When NOT to use useEffect

If a value can be calculated from props or state, **do not** use an effect for it:

✗ **An unnecessary effect and an extra render**

```
const [fullName, setFullName] = useState("");
useEffect(() => {
  setFullName(first + " " + last);
}, [first, last]);
```

✓ **Just calculate it during render**

```
const fullName = first + " " + last;
```

Likewise, code that responds to a click belongs in the click handler, not in an effect.

Practice Task

1. Log "Component mounted" exactly once, and log "Name changed to ___" every time a name input changes.
2. **Auto-saving note:** a `<textarea>` whose text is saved to `localStorage` whenever it changes. When the page reloads, the saved note should appear. (Hint: `useState(() => localStorage.getItem("note") ?? "")`.)
3. **Window width:** show the current `window.innerWidth`, and update it when the window is resized. Add a `resize` listener in an effect, and remove it in the cleanup.

Solution

Exercise 1

```
import { useState, useEffect } from "react";

export default function NameLogger() {
  const [name, setName] = useState("");

  useEffect(() => {
    console.log("Component mounted");
  }, []);

  useEffect(() => {
    if (name) console.log("Name changed to", name);
  }, [name]);

  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}
```

Exercise 2

```
import { useState, useEffect } from "react";

export default function Note() {
  const [note, setNote] = useState(() => localStorage.getItem("note") ?? "");

  useEffect(() => {
    localStorage.setItem("note", note);
  }, [note]);

  return <textarea value={note} onChange={(e) => setNote(e.target.value)} />;
}
```

Passing a **function** to `useState` (a "lazy initializer") means `localStorage` is read only on the first render, not on every render.

Exercise 3

```
import { useState, useEffect } from "react";

export default function WindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    function handleResize() {
      setWidth(window.innerWidth);
    }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return <p>Window width: {width}px</p>;
}
```

Common Mistakes

COMMON MISTAKE · Missing dependencies

If an effect uses a value such as `userId` but the array is `[]`, the effect only ever sees the first value. List every prop and state value the effect uses. The ESLint rule `react-hooks/exhaustive-deps`, included in the Vite template, warns you about this.

COMMON MISTAKE · Infinite loops

× No dependency array + setting state = an endless loop

```
useEffect(() => {
  setCount(count + 1);
});
```

Every render runs the effect, and the effect causes another render. Add the correct dependency array, or ask yourself whether you need an effect at all.

COMMON MISTAKE · Forgetting cleanup

Intervals, event listeners and subscriptions keep running after a component disappears unless you remove them in the cleanup function.

COMMON MISTAKE · Making the effect callback async

`useEffect(async () => { ... })` is not allowed, because an async function returns a Promise, and React expects either nothing or a cleanup function. Chapter 12 shows the correct pattern.

Quick Check

1. What is a side effect? Give two examples.
2. What is the difference between `useEffect(fn)`, `useEffect(fn, [])` and `useEffect(fn, [count])`?
3. When does the cleanup function run?
4. Why do effects run twice on the first load in development?
5. Should you use an effect to calculate `fullName` from `first` and `last`?

CHAPTER 12

API Calls / Fetch

What is it?

An **API call** asks a server for data over HTTP. In React, you use `fetch()` (which you already know) inside `useEffect`, and you store the result in state so it appears on the screen.

Why do we need it?

Real applications do not hard-code their data. Products, users and orders live on a server. Your components must request that data, wait for it, handle failures, and then display it.

```
Component renders (loading = true)
  ↓
useEffect runs
  ↓
fetch() sends the request
  ↓
API (server) responds
  ↓
response.json() parses the JSON
  ↓
setState() stores the data
  ↓
Component re-renders → UI shows the data
```

Syntax

Every data-loading component needs **three pieces of state**: the data, whether it is loading, and any error.

```
const [data, setData] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  async function loadData() {
    try {
      const response = await fetch(URL);
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      const json = await response.json();
      setData(json);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  loadData();
}, []);
```

The effect callback cannot be `async` itself, so we **define** an `async` function inside it and **call** it immediately.

Small Example

We use **JSONPlaceholder**, a free practice API that needs no sign-up.

src/UserList.jsx

```
import { useState, useEffect } from "react";

const URL = "https://jsonplaceholder.typicode.com/users";

export default function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadUsers() {
      try {
        const response = await fetch(URL);
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        const data = await response.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadUsers();
  }, []);

  if (loading) return <p>Loading users...</p>;
  if (error) return <p style={{ color: "red" }}>Error: {error}</p>;

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>
          <strong>{user.name}</strong> - {user.email}
        </li>
      ))}
    </ul>
  );
}
```

The API returns JSON like this (shortened):

Part of the response from /users

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "email": "Sincere@april.biz",
    "address": { "city": "Gwenborough", ... },
    "phone": "1-770-736-8031 x56442",
    ...
  },
  ...
]
```

Explain the Code

- `loading` starts as `true` because the request begins immediately.
- `await fetch(URL)` waits for the server's response.
- `response.ok` is `true` for status codes 200–299. `fetch` **does not throw on 404 or 500**, so you must check this yourself.
- `await response.json()` turns the response body into a JavaScript array.
- `finally` runs whether the request succeeded or failed, so `loading` always ends.
- The early returns show the right screen for each situation: loading, error, or data.
- The empty array `[]` means "fetch once, when the component first appears".

What Happens?

Render 1	<code>loading = true</code>	→ shows "Loading users..."
	effect runs	→ the request leaves the browser
	... the network takes some time ...	
	response arrives	→ <code>setUsers(data)</code> and <code>setLoading(false)</code>
Render 2	<code>loading = false</code>	→ shows the list of 10 users

React groups (**batches**) the state updates made together, so `setUsers` and `setLoading` produce one re-render, not two.

TIP · Test your error state

Change the URL to `.../usersXYZ` and reload. You should see "Error: HTTP 404". Always check that your error screen actually works.

Fetching again when a value changes

When the URL depends on a prop or state, such as a user ID, put it in the dependency array. Add an **AbortController** so an old, slow request cannot overwrite newer data:

```
useEffect(() => {
  const controller = new AbortController();

  async function loadUser() {
    setLoading(true);
    setError(null);
    try {
      const res = await fetch(
        `https://jsonplaceholder.typicode.com/users/${userId}`,
        { signal: controller.signal }
      );
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setUser(await res.json());
    } catch (err) {
      if (err.name !== "AbortError") setError(err.message);
    } finally {
      if (!controller.signal.aborted) setLoading(false);
    }
  }

  loadUser();
  return () => controller.abort();
}, [userId]);
```

If `userId` changes from 1 to 2 before user 1 has loaded, the cleanup **aborts** the first request. Only the latest request can update the screen.

Sending data (POST)

Sending data usually happens in an **event handler** (for example, on form submit), not in an effect:

```
async function createPost(title) {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, body: "Hello", userId: 1 }),
  });
  const saved = await res.json();
  console.log("Server returned:", saved); // JSONPlaceholder pretends to save it
}
```

Practice Task

1. **Fetch users:** extend `UserList` to also show each user's phone number and city (`user.address.city`).
2. **Fetch products:** load products from `https://dummyjson.com/products?limit=10`. This API returns an **object**, `{ products: [...], total, skip, limit }`, so the array is in `json.products`. Show each product's thumbnail image, title and price.
3. **Loading, error and reload:** add a "Reload" button that fetches the products again. Show "Loading products..." during every load, and a red error message if the request fails. (Hint: a `reloadKey` number in state, used as a dependency.)

Solution

Exercise 1 (the list item inside `UserList`)

```
<li key={user.id}>
  <strong>{user.name}</strong> - {user.email} - {user.phone} -{" "}
  {user.address.city}
</li>
```

`{" "}` inserts a space where JSX would otherwise trim it at the end of a line.

Exercises 2 and 3: src/ProductList.jsx

```
import { useState, useEffect } from "react";

const URL = "https://dummyjson.com/products?limit=10";

export default function ProductList() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [reloadKey, setReloadKey] = useState(0);

  useEffect(() => {
    async function loadProducts() {
      setLoading(true);
      setError(null);
      try {
        const res = await fetch(URL);
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const json = await res.json();
        setProducts(json.products);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadProducts();
  }, [reloadKey]);

  return (
    <div>
      <button onClick={() => setReloadKey((k) => k + 1)}>Reload</button>

      {loading && <p>Loading products...</p>}
      {error && <p style={{ color: "red" }}>Error: {error}</p>}

      {!loading && !error && (
        <ul>
          {products.map((p) => (
            <li key={p.id}>
              <img src={p.thumbnail} alt={p.title} width="50" />
              {p.title} - ${p.price}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

Common Mistakes

COMMON MISTAKE · Not checking response.ok

A 404 or 500 response does not go to `catch`. Without `if (!res.ok) throw ...`, your code tries to use an error page as if it were data.

COMMON MISTAKE · Getting the response shape wrong

"products.map is not a function" usually means you stored the whole object `{ products: [...] }` instead of the array `json.products`. `console.log` the JSON once to see its shape.

COMMON MISTAKE · No loading state

If `users` starts as `undefined` and you render `users.map(...)` before the data arrives, the app crashes. Start with a sensible initial value (`[]` or `null`) and handle loading explicitly.

COMMON MISTAKE · Fetching in the component body

Calling `fetch` outside `useEffect` sends a request on **every** render, and setting state from it causes another render: an endless stream of requests.

Quick Check

1. What three pieces of state does a typical data-loading component need?
2. Why can't the effect callback itself be `async`?
3. Does `fetch` throw an error for a 404 response?
4. What does the dependency array `[userId]` do for a fetch?
5. What problem does `AbortController` solve?

CHAPTER 13

React Router

What is it?

React Router is a library that connects **URLs** to **components**. It lets a single-page React app have several "pages" (`/`, `/about`, `/products/5`) without reloading the browser.

Why do we need it?

A React app is one HTML page. Without routing, every screen would live at the same address, so users could not bookmark a product, share a link, or use the browser's Back button. React Router gives each screen its own URL, while keeping the app fast because the page never fully reloads.

URL	Component shown
<code>/</code>	Home
<code>/about</code>	About
<code>/products</code>	Products
<code>/products/:id</code>	ProductDetails (<code>:id</code> is a placeholder, e.g. <code>/products/2</code>)
anything else	NotFound

Syntax

Install the package in your project:

Terminal

```
npm install react-router
```

NOTE • Versions

This workbook uses **React Router 8** (released June 2026), where everything is imported from `"react-router"`. Many tutorials use version 6 and import from `"react-router-dom"`. That package no longer exists in version 8, so change the import to `"react-router"`; the components below work the same way in versions 6, 7 and 8. Very old tutorials (version 5) use `<Switch>` and `component=`; those no longer exist at all.

Piece	What it does
<code><BrowserRouter></code>	Turns routing on. Wrap your whole app in it, once.
<code><Routes></code>	A container that picks the one <code><Route></code> matching the current URL.
<code><Route path="..." element={...} /></code>	"When the URL is this path, show this element."
<code><Link to="..."></code>	A link that changes the URL without reloading the page.
<code>useNavigate()</code>	Returns a function to change the page from code, e.g. after a form submit.
<code>useParams()</code>	Reads URL parameters such as <code>:id</code> .

Small Example

First, wrap the app in **BrowserRouter**:

src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

Some local data to display:

src/data.js

```
export const products = [
  { id: 1, name: "Keyboard", price: 45, description: "Mechanical, blue switches." },
  { id: 2, name: "Mouse", price: 20, description: "Wireless, silent clicks." },
  { id: 3, name: "Monitor", price: 180, description: "27 inch, 1440p." },
];
```

The routes and the navigation bar:

src/App.jsx

```
import { Routes, Route, Link } from "react-router";
import Home from "./pages/Home.jsx";
import About from "./pages/About.jsx";
import Products from "./pages/Products.jsx";
import ProductDetails from "./pages/ProductDetails.jsx";
import NotFound from "./pages/NotFound.jsx";

export default function App() {
  return (
    <div>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link> |{" "}
        <Link to="/products">Products</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/products" element={<Products />} />
        <Route path="/products/:id" element={<ProductDetails />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </div>
  );
}
```

The pages (each in **src/pages/**):

src/pages/Home.jsx, About.jsx and NotFound.jsx

```
// Home.jsx
export default function Home() {
  return <h1>Welcome to the shop</h1>;
}

// About.jsx
export default function About() {
  return <h1>About us</h1>;
}

// NotFound.jsx
import { Link } from "react-router";

export default function NotFound() {
  return (
    <div>
      <h1>404: Page not found</h1>
      <Link to="/">Go home</Link>
    </div>
  );
}
```

src/pages/Products.jsx

```
import { Link } from "react-router";
import { products } from "../data.js";

export default function Products() {
  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>
          <Link to={`/${products}/${p.id}`}>{p.name}</Link> - ${p.price}
        </li>
      ))}
    </ul>
  );
}
```

src/pages/ProductDetails.jsx

```
import { useParams, useNavigate } from "react-router";
import { products } from "../data.js";

export default function ProductDetails() {
  const { id } = useParams();
  const navigate = useNavigate();
  const product = products.find((p) => p.id === Number(id));

  if (!product) return <p>Product not found.</p>;

  return (
    <div>
      <h2>{product.name}</h2>
      <p>{product.description}</p>
      <p>Price: ${product.price}</p>
      <button onClick={() => navigate("/products")}>All products</button>
      <button onClick={() => navigate(-1)}>Go back</button>
    </div>
  );
}
```

Explain the Code

- `<Route path="/products/:id" ...>`: the colon marks `id` as a **parameter**. It matches `/products/1`, `/products/2`, and so on.
- `useParams()` returns `{ id: "2" }` for `/products/2`. URL parameters are always **strings**, so we convert with `Number(id)` before comparing.
- `path="*" matches anything not matched by another route: your 404 page.`
- `<Link to=...>` replaces `<a href=...>`.
- `navigate("/products")` moves to a page from code; `navigate(-1)` goes back one step, like the browser's Back button.
- The `<nav>` is outside `<Routes>`, so it stays visible on every page.

What Happens?

1. The user clicks a `<Link>`. React Router changes the address bar using the browser's History API, **without** a page reload.
2. React Router notices the new URL and checks each `<Route>`'s `path`.
3. `<Routes>` renders the element of the matching route. Everything outside `<Routes>` stays as it is, and all state in `App` is kept.

COMMON MISTAKE · Using a normal `a href`

`<a href="/products">` makes the browser download the whole page again. Your app restarts and all state is lost. Use `<Link to="/products">` for links inside your app. Normal `<a>` tags are still correct for **external** websites.

Practice Task

1. Add a **Contact** page at `/contact` and a link to it in the navigation bar.
2. In `ProductDetails`, add a "Next product" button that goes to the next ID, or back to the first product after the last one.
3. Replace the `Links` in the navigation bar with `NavLink`, which lets you style the link of the current page. Give the active link the class `active`. (Hint: `className={({ isActive }) => (isActive ? "active" : "")}`.)

Solution

Exercise 1

```
// src/pages/Contact.jsx
export default function Contact() {
  return <h1>Contact: hello@shop.example</h1>;
}

// In App.jsx: import Contact, then add
<Link to="/contact">Contact</Link>
<Route path="/contact" element={<Contact />} />
```

Exercise 2 (inside `ProductDetails`, after the product is found)

```
const index = products.findIndex((p) => p.id === product.id);
const next = products[(index + 1) % products.length];

<button onClick={() => navigate(`/products/${next.id}`)}>Next product</button>
```

% `products.length` wraps around to index 0 after the last product.

Exercise 3

```
import { NavLink } from "react-router";

const linkClass = ({ isActive }) => (isActive ? "active" : "");

<nav>
  <NavLink to="/" end className={linkClass}>Home</NavLink> |{" "}
  <NavLink to="/about" className={linkClass}>About</NavLink> |{" "}
  <NavLink to="/products" className={linkClass}>Products</NavLink>
</nav>

/* index.css */
/* .active { font-weight: bold; color: #087ea4; } */
```

The `end` prop on the Home link means "only active on exactly `/`". Without it, Home would look active on every page, because every path starts with `/`.

Common Mistakes

COMMON MISTAKE · Forgetting `BrowserRouter`

An error such as "useNavigate() may be used only in the context of a Router component" means the app is not wrapped in `<BrowserRouter>`.

COMMON MISTAKE · Comparing a string ID with a number

`products.find(p => p.id === id)` finds nothing, because `id` from the URL is `"2"`, not `2`. Use `Number(id)`.

COMMON MISTAKE · Copying code from old tutorials

`<Switch>`, `component={Home}` and `useHistory()` are from React Router 5 and no longer exist. Use `<Routes>`, `element={<Home />}` and `useNavigate()`.

Quick Check

1. Why do single-page apps need a router?
2. What is the difference between `<Link>` and `<a href>`?
3. How do you read `:id` from the URL, and what type is it?
4. What does `path="*" match`?
5. When would you use `useNavigate()` instead of `<Link>`?

CHAPTER 14

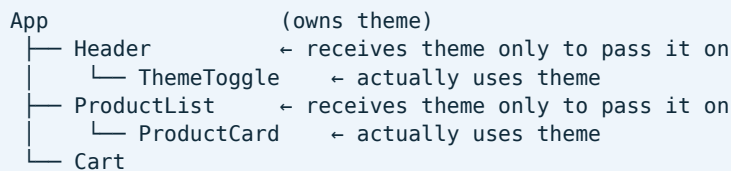
Context API

What is it?

Context lets a component share data with **every component below it**, at any depth, **without** passing props through each level in between.

Why do we need it?

Imagine the current theme ("light" or "dark") is state in **App**, but only **ThemeToggle** and **ProductCard**, deep in the tree, actually use it:



x Prop drilling: Header does not care about theme, it just passes it on

```
function App() {
  const [theme, setTheme] = useState("light");
  return <Header theme={theme} setTheme={setTheme} />;
}

function Header({ theme, setTheme }) {
  return <ThemeToggle theme={theme} setTheme={setTheme} />;
}
```

Passing props through components that do not use them is called **prop drilling**. With three or four levels, it becomes tedious and fragile. Context lets **ThemeToggle** read the theme directly.

Syntax

```
import { createContext, useContext } from "react";

// 1. Create the context (usually in its own file)
const ThemeContext = createContext(null);

// 2. Provide a value to everything inside
<ThemeContext.Provider value={theme}>
  <App />
</ThemeContext.Provider>

// 3. Read it in any component below
const theme = useContext(ThemeContext);
```

NOTE · React 19 shorthand

In React 19 you can also write `<ThemeContext value={theme}>` without `.Provider`. This workbook uses `.Provider` because it works in all recent versions and in most tutorials you will find.

Small Example

```
App
├── Header      (reads and changes the theme)
├── ProductList (reads the theme)
└── Cart        (does not use the theme at all)
```

A common pattern is to put the context **and** a provider component in one file:

src/ThemeContext.jsx

```
import { createContext, useState } from "react";

export const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  function toggleTheme() {
    setTheme((t) => (t === "light" ? "dark" : "light"));
  }

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

src/App.jsx

```
import { useContext } from "react";
import { ThemeContext, ThemeProvider } from "../ThemeContext.jsx";

function Header() {
  const { theme, toggleTheme } = useContext(ThemeContext);
  return (
    <header>
      <h1>My Shop</h1>
      <button onClick={toggleTheme}>
        Switch to {theme === "light" ? "dark" : "light"} mode
      </button>
    </header>
  );
}

function ProductList() {
  const { theme } = useContext(ThemeContext);
  const style =
    theme === "dark"
      ? { background: "#222", color: "#fff", padding: 16 }
      : { background: "#fff", color: "#222", padding: 16 };

  return <div style={style}>Products appear here ({theme} mode)</div>;
}

function Cart() {
  return <p>Cart (does not care about the theme)</p>;
}

export default function App() {
  return (
    <ThemeProvider>
      <Header />
      <ProductList />
      <Cart />
    </ThemeProvider>
  );
}
```

NOTE · A harmless warning

Vite's lint setup may warn that a file exporting both a component and a non-component "breaks Fast Refresh". It only affects instant reloading during development. You can ignore it while learning, or later move the context into its own file.

Explain the Code

- `createContext(null)` creates the context. `null` is the value used if a component reads it **outside** any provider.
- `ThemeProvider` owns the state and passes `{ theme, toggleTheme }` as the context value.
- `{children}` means "everything I wrap can access this context".
- `useContext(ThemeContext)` in `Header` and `ProductList` reads the value directly: no props needed.
- `Cart` does not call `useContext`, so it is not involved at all.

What Happens?

```
ThemeProvider  value = { theme: "light", toggleTheme }
|
├── Header      useContext → gets the value
├── ProductList  useContext → gets the value
└── Cart        (does not read the context)

└── re-render when the value changes
    └── Cart is not affected
```

When `toggleTheme` runs, the provider's state changes, so its value changes. React then re-renders **every component that reads this context**. A component always reads the value of the **nearest** provider above it.

When NOT to use Context

- **Data needed only one or two levels down.** Plain props are simpler and make the data flow obvious.
- **Values that change very often**, such as the text of an input, shared across a large tree. Every consumer re-renders on every change.
- **As a replacement for all state.** Components that depend on context are harder to reuse elsewhere.

Good candidates for context: the **theme**, the **logged-in user**, the **language**, and a **shopping cart**.

Practice Task

1. **UserContext:** create a context holding `user` (`null` or `{ name }`) with `login(name)` and `logout()` functions. `Header` shows "Hi, (name)" with a "Log out" button, or a "Log in as Asha" button when nobody is logged in.
2. **CartContext:** create a context with `items` and `addToCart(product)`. Each product in `ProductList` gets an "Add" button, and `Cart` shows how many items it contains.
3. Inside `ThemeProvider`, use `useEffect` to set `document.body.style.background` to match the current theme.

Solution

Exercise 1: src/UserContext.jsx

```
import { createContext, useState } from "react";

export const UserContext = createContext(null);

export function UserProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = (name) => setUser({ name });
  const logout = () => setUser(null);

  return (
    <UserContext.Provider value={{ user, login, logout }}>
      {children}
    </UserContext.Provider>
  );
}

// In Header:
// const { user, login, logout } = useContext(UserContext);
// return user ? (
//   <p>Hi, {user.name} <button onClick={logout}>Log out</button></p>
// ) : (
//   <button onClick={() => login("Asha")}>Log in as Asha</button>
// );
```

Exercise 2: src/CartContext.jsx and its use

```
import { createContext, useContext, useState } from "react";

export const CartContext = createContext(null);

export function CartProvider({ children }) {
  const [items, setItems] = useState([]);
  const addToCart = (product) => setItems((prev) => [...prev, product]);

  return (
    <CartContext.Provider value={{ items, addToCart }}>
      {children}
    </CartContext.Provider>
  );
}

// In ProductList:
function ProductList() {
  const { addToCart } = useContext(CartContext);
  const products = [{ id: 1, name: "Pen" }, { id: 2, name: "Book" }];
  return products.map((p) => (
    <p key={p.id}>
      {p.name} <button onClick={() => addToCart(p)}>Add</button>
    </p>
  ));
}

// In Cart:
function Cart() {
  const { items } = useContext(CartContext);
  return <p>Cart: {items.length} items</p>;
}
```

Exercise 3 (inside ThemeProvider, before the return)

```
useEffect(() => {
  document.body.style.background = theme === "dark" ? "#111" : "#fff";
}, [theme]);
```

Remember to add `useEffect` to the import from `"react"`.

Common Mistakes

COMMON MISTAKE · Using a context outside its provider

If `Header` is not inside `<ThemeProvider>`, `useContext` returns the default `null`, and `const { theme } = null` crashes. Make sure the provider wraps every component that reads the context. Chapter 15 shows a hook that gives a clear error message for this.

COMMON MISTAKE · Forgetting to export

If the context object is not exported, other files cannot import it for `useContext`.

COMMON MISTAKE · One giant context for everything

Putting the user, theme, cart and filters in one context makes every consumer re-render when anything changes. Create a separate context for each concern.

Quick Check

1. What is prop drilling?
2. What are the three steps to use Context?
3. What does `useContext` return if there is no provider above the component?
4. Which components re-render when a context value changes?
5. Name one situation where plain props are better than Context.

CHAPTER 15

Custom Hooks

What is it?

A **hook** is a special function whose name starts with **use** and that lets a component use React features. You already know three built-in hooks: **useState**, **useEffect** and **useContext**.

A **custom hook** is **your own** function, named **useSomething**, that calls other hooks inside it. It lets you package **stateful logic** once and reuse it in many components.

Why do we need it?

Look back at Chapter 12. Every component that loads data repeats the same 20 lines: three **useState** calls, a **useEffect**, **try/catch/finally**. If ten components fetch data, that is 200 lines of copy-paste, and a bug fix must be made ten times.

A custom hook moves that logic into one function. Each component then needs a single line.

Syntax

```
function useSomething(input) {  
  const [value, setValue] = useState(/* ... */);  
  useEffect(() => { /* ... */ }, [input]);  
  return value; // return anything: a value, an array, an object  
}
```

Three rules:

- The name **must start with use**, so React and the linter know the Rules of Hooks apply.
- Call it at the **top level** of a component or another hook, never inside **if**, loops or regular functions.
- It returns **data and functions**, not JSX.

Small Example

We build three hooks, from very simple to realistic.

Step 1: useToggle

src/hooks/useToggle.js

```
import { useState } from "react";  
  
export function useToggle(initial = false) {  
  const [on, setOn] = useState(initial);  
  const toggle = () => setOn((v) => !v);  
  return [on, toggle];  
}
```

Using it

```
const [isOpen, toggleOpen] = useToggle();  
const [isDark, toggleDark] = useToggle(true);  
  
<button onClick={toggleOpen}>{isOpen ? "Close" : "Open"} menu</button>
```

Hook files contain no JSX, so they can use the `.js` extension.

Step 2: useCounter

src/hooks/useCounter.js

```
import { useState } from "react";

export function useCounter(initial = 0, min = -Infinity, max = Infinity) {
  const [count, setCount] = useState(initial);

  const increment = () => setCount((c) => Math.min(c + 1, max));
  const decrement = () => setCount((c) => Math.max(c - 1, min));
  const reset = () => setCount(initial);

  return { count, increment, decrement, reset };
}
```

src/TicketBooking.jsx

```
import { useCounter } from "../hooks/useCounter.js";

export default function TicketBooking() {
  const adults = useCounter(1, 1, 10);
  const children = useCounter(0, 0, 10);

  return (
    <div>
      <p>
        Adults: <button onClick={adults.decrement}>-</button> {adults.count}{" "}
        <button onClick={adults.increment}>+</button>
      </p>
      <p>
        Children: <button onClick={children.decrement}>-</button> {children.count}{" "}
        <button onClick={children.increment}>+</button>
      </p>
      <p>Total tickets: {adults.count + children.count}</p>
    </div>
  );
}
```

Step 3: useFetch

src/hooks/useFetch.js

```
import { useState, useEffect } from "react";

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    async function load() {
      setLoading(true);
      setError(null);
      try {
        const res = await fetch(url, { signal: controller.signal });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        setData(await res.json());
      } catch (err) {
        if (err.name !== "AbortError") setError(err.message);
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }

    load();
    return () => controller.abort();
  }, [url]);

  return { data, loading, error };
}
```

Now the `UserList` from Chapter 12 shrinks to this:

src/UserList.jsx

```
import { useFetch } from "../hooks/useFetch.js";

export default function UserList() {
  const { data: users, loading, error } = useFetch(
    "https://jsonplaceholder.typicode.com/users"
  );

  if (loading) return <p>Loading users...</p>;
  if (error) return <p style={{ color: "red" }}>Error: {error}</p>;

  return (
    <ul>
      {users.map((u) => (
        <li key={u.id}>{u.name}</li>
      ))}
    </ul>
  );
}
```

Explain the Code

- `useToggle` wraps one `useState` and gives back a nicer API: the value and a `toggle` function.
- `useCounter` returns an **object**, so the caller can pick names like `adults.count`. Min and max limits live inside the hook, so no component can forget them.
- `useCounter` is called **twice** in `TicketBooking`. Each call has its **own** separate state.

- `useFetch` contains everything from Chapter 12: three states, the effect, error handling and the abort cleanup. It re-fetches whenever `url` changes.
- `const { data: users, ... } renames data to users while destructuring.`

What Happens?

A custom hook is not magic: React simply runs its code as if you had written it **inside** the component. That is why each component that calls a hook gets its **own** copy of the state:

```
UserList → useFetch(usersUrl) → its own data / loading / error
PostList → useFetch(postsUrl) → its own data / loading / error
```

REMEMBER

Custom hooks share **logic**, not **data**, and not **UI**. A hook returns values and functions; a component returns JSX. If several components need the **same data**, combine a hook with Context.

A hook for context

This small hook makes Context safer to use. Put it in the same file as the context:

`src/ThemeContext.jsx` (added at the bottom)

```
import { useContext } from "react";

export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error("useTheme must be used inside <ThemeProvider>");
  }
  return context;
}

// Components now write: const { theme, toggleTheme } = useTheme();
```

Practice Task

1. **`useLocalStorage(key, initialValue)`**: works exactly like `useState`, but also saves the value to `localStorage` and loads it on the first render. Store the value as JSON.
2. **`useWindowWidth()`**: returns the current window width, using your solution from Chapter 11.
3. Use `useFetch` to show product titles from <https://dummyjson.com/products?limit=5>. Remember that the array is in `data.products`.

Solution

Exercise 1: src/hooks/useLocalStorage.js

```
import { useState, useEffect } from "react";

export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved !== null ? JSON.parse(saved) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Usage: const [name, setName] = useLocalStorage("name", "");
```

Exercise 2: src/hooks/useWindowWidth.js

```
import { useState, useEffect } from "react";

export function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const onResize = () => setWidth(window.innerWidth);
    window.addEventListener("resize", onResize);
    return () => window.removeEventListener("resize", onResize);
  }, []);

  return width;
}
```

Exercise 3

```
import { useFetch } from "../hooks/useFetch.js";

export default function TopProducts() {
  const { data, loading, error } = useFetch(
    "https://dummyjson.com/products?limit=5"
  );

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ol>
      {data.products.map((p) => (
        <li key={p.id}>{p.title}</li>
      ))}
    </ol>
  );
}
```

Common Mistakes

COMMON MISTAKE · Not starting the name with use

A function called `fetchData()` that calls `useState` breaks the Rules of Hooks, and the linter cannot check it. Name it `useFetchData()`.

COMMON MISTAKE · Expecting two components to share state

Two components calling `useCounter()` get two independent counters. To share one value, lift the state up or use Context.

COMMON MISTAKE · Returning JSX from a hook

If your function returns JSX, it is a component, not a hook. Hooks return data and functions.

COMMON MISTAKE · Passing a new object every render

`useFetch(url, { headers: {...} })` with an options object in the dependency array creates a "new" object on every render, so the effect runs forever. Keep dependencies simple (strings and numbers), or define constant objects outside the component.

Quick Check

1. What makes a function a custom hook?
2. If two components call `useFetch` with the same URL, do they share the data?
3. What is the difference between a custom hook and a component?
4. Why must a custom hook's name start with `use`?

Before You Build: Thinking in React

Every React project, big or small, can be planned with the same five steps. Use them for each mini project before you look at the code.

1. **Sketch the screen** and draw a box around each part. Each box becomes a component.
2. **Draw the component tree**: which component contains which?
3. **Build a static version first**, with hard-coded data and no state. Just get the layout right.
4. **Find the minimum state**. Ask of every piece of data: does it change? Can it be calculated from something else? Only data that changes and cannot be calculated becomes state.
5. **Decide where each piece of state lives** (the closest common parent of everything that uses it), then add event handlers that flow back up as callback props.

TIP · How to use the projects

Read the requirements, the tree and the state plan. Then **close the book and try to build it yourself**. Use the code only when you are stuck, or to compare with your version afterwards. There is always more than one correct solution.

Each project starts from a fresh Vite project (see "Setting Up Your First React Project"), with `App.css` and `index.css` emptied.

MINI PROJECT 1

Todo App

What you will build

A classic todo list where you can add tasks, mark them as done, delete them and filter them.

Concept	Where it is used
Components and props	<code>TodoForm</code> , <code>FilterBar</code> , <code>TodoList</code> , <code>TodoItem</code>
State	the list of todos, the current filter, the form text
Events	submit, checkbox change, delete click, filter click
Lists and keys	rendering todos with <code>.map()</code>
Conditional rendering	the empty state, the "done" styling, "task" vs "tasks"
Forms	a controlled input with simple validation

Features: add a task (empty text is rejected with a message), tick a task as done, delete a task, filter by All / Active / Done, show how many tasks are left, and show a friendly message when the list is empty.

Component tree and state plan

```

App
├── TodoForm      state: text, error      calls onAdd(text)
├── FilterBar     props: filter           calls onChange(filter)
├── TodoList      props: todos (already filtered)
│   └── TodoItem  props: todo             calls onToggle(id), onDelete(id)
└── (footer text) calculated: remaining count
  
```

Data	State?	Lives in	Why
All todos	Yes	<code>App</code>	<code>TodoForm</code> adds to it and <code>TodoList</code> shows it: common parent
Current filter	Yes	<code>App</code>	<code>FilterBar</code> changes it and <code>App</code> uses it to filter
Input text	Yes	<code>TodoForm</code>	Only the form needs it
Visible todos	No	calculated in <code>App</code>	Can be derived from <code>todos</code> + <code>filter</code>
Remaining count	No	calculated in <code>App</code>	Can be derived from <code>todos</code>

The code

src/ToDoForm.jsx

```
import { useState } from "react";

export default function TodoForm({ onAdd }) {
  const [text, setText] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    if (!text.trim()) {
      setError("Please type a task first.");
      return;
    }
    onAdd(text.trim());
    setText("");
    setError("");
  }

  return (
    <form onSubmit={handleSubmit} className="todo-form">
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="What needs to be done?"
      />
      <button type="submit">Add</button>
      {error && <p className="error">{error}</p>}
    </form>
  );
}
```

src/FilterBar.jsx

```
const FILTERS = ["all", "active", "done"];

export default function FilterBar({ filter, onChange }) {
  return (
    <div className="filters">
      {FILTERS.map((f) => (
        <button
          key={f}
          className={f === filter ? "active" : ""}
          onClick={() => onChange(f)}
        >
          {f}
        </button>
      ))}
    </div>
  );
}
```

src/ToDoItem.jsx

```
export default function ToDoItem({ todo, onToggle, onDelete }) {
  return (
    <li className={todo.done ? "todo done" : "todo"}>
      <input
        type="checkbox"
        checked={todo.done}
        onChange={() => onToggle(todo.id)}
      />
      <span>{todo.text}</span>
      <button onClick={() => onDelete(todo.id)}>Delete</button>
    </li>
  );
}
```

src/ToDoList.jsx

```
import ToDoItem from "../ToDoItem.jsx";

export default function ToDoList({ todos, onToggle, onDelete }) {
  if (todos.length === 0) {
    return <p className="empty">Nothing here. Add a task!</p>;
  }

  return (
    <ul className="todo-list">
      {todos.map((todo) => (
        <ToDoItem
          key={todo.id}
          todo={todo}
          onToggle={onToggle}
          onDelete={onDelete}
        />
      ))}
    </ul>
  );
}
```

src/App.jsx

```

import { useState } from "react";
import TodoForm from "../TodoForm.jsx";
import FilterBar from "../FilterBar.jsx";
import TodoList from "../TodoList.jsx";

export default function App() {
  const [todos, setTodos] = useState([
    { id: 1, text: "Learn components", done: true },
    { id: 2, text: "Learn state", done: false },
  ]);
  const [filter, setFilter] = useState("all");

  function addTodo(text) {
    setTodos((prev) => [...prev, { id: Date.now(), text, done: false }]);
  }

  function toggleTodo(id) {
    setTodos((prev) =>
      prev.map((t) => (t.id === id ? { ...t, done: !t.done } : t))
    );
  }

  function deleteTodo(id) {
    setTodos((prev) => prev.filter((t) => t.id !== id));
  }

  const visibleTodos = todos.filter((t) => {
    if (filter === "active") return !t.done;
    if (filter === "done") return t.done;
    return true;
  });

  const remaining = todos.filter((t) => !t.done).length;

  return (
    <div className="todo-app">
      <h1>Todo App</h1>
      <TodoForm onAdd={addTodo} />
      <FilterBar filter={filter} onChange={setFilter} />
      <TodoList
        todos={visibleTodos}
        onToggle={toggleTodo}
        onDelete={deleteTodo}
      />
      <p className="footer">
        {remaining} {remaining === 1 ? "task" : "tasks"} left
      </p>
    </div>
  );
}

```

src/index.css

```

body { font-family: system-ui, sans-serif; background: #f4f6f8; }
.todo-app { max-width: 480px; margin: 40px auto; background: #fff;
  padding: 24px; border-radius: 12px; }
.todo-form { display: flex; gap: 8px; flex-wrap: wrap; }
.todo-form input { flex: 1; padding: 8px; }
.error { color: #cf222e; width: 100%; margin: 4px 0 0; }
.filters { margin: 16px 0; display: flex; gap: 6px; }
.filters .active { background: #087ea4; color: #fff; }
.todo-list { list-style: none; padding: 0; }
.todo { display: flex; gap: 8px; align-items: center; padding: 6px 0; }
.todo span { flex: 1; }
.todo.done span { text-decoration: line-through; color: #888; }
.empty, .footer { color: #666; }

```

Key ideas to notice

- `toggleTodo` uses `.map()` to create a new array where only one todo is replaced with a copy: `{ ...t, done: !t.done }`. Nothing is mutated.
- `FilterBar` receives `setFilter` directly as `onChange`. A setter is just a function, so you can pass it as a callback prop.
- The filtered list is calculated on every render, never stored. Change the filter, and the list follows automatically.

TIP · Extend it

Add a "Clear completed" button; allow editing a task with a double-click; save the todos to `localStorage` with the `useLocalStorage` hook from Chapter 15.

MINI PROJECT 2

Product Explorer

What you will build

A small shop catalogue that loads real data from an API. Users can search and filter products, open a details page, and go back.

Concept	Where it is used
Components and props	<code>Navbar</code> , <code>SearchBar</code> , <code>ProductCard</code> , two page components
State	products, loading, error, search text, selected category
<code>.map()</code> and keys	the product grid and the category dropdown
<code>useEffect</code> and API calls	loading the product list and a single product
Loading and error states	on both pages
React Router	<code>/</code> for the list and <code>/products/:id</code> for the details

API used: DummyJSON, a free practice API.

Request	Returns
<code>https://dummyjson.com/products?limit=30</code>	<code>{ products: [...], total, skip, limit }</code>
<code>https://dummyjson.com/products/5</code>	one product object

Each product has (among other fields) `id`, `title`, `description`, `price`, `category`, `rating`, `thumbnail`, `images` and `brand` (some products have no brand).

Component tree

```
BrowserRouter (in main.jsx)
├── App
│   ├── Navbar
│   └── Routes
│       ├── "/" → ProductsPage state: products, loading, error,
│           │   ├── SearchBar search, category
│           └── ProductCard × N
│       └── "/products/:id" → ProductDetailsPage state: product, loading, error
```

The code

Install the router first: `npm install react-router`.

src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router";
import App from "../App.jsx";
import "../index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

src/App.jsx

```
import { Routes, Route } from "react-router";
import Navbar from "../components/Navbar.jsx";
import ProductsPage from "../pages/ProductsPage.jsx";
import ProductDetailsPage from "../pages/ProductDetailsPage.jsx";

export default function App() {
  return (
    <>
      <Navbar />
      <main className="container">
        <Routes>
          <Route path="/" element={<ProductsPage />} />
          <Route path="/products/:id" element={<ProductDetailsPage />} />
          <Route path="*" element={<p>Page not found.</p>} />
        </Routes>
      </main>
    </>
  );
}
```

src/components/Navbar.jsx

```
import { Link } from "react-router";

export default function Navbar() {
  return (
    <nav className="navbar">
      <Link to="/">Product Explorer</Link>
    </nav>
  );
}
```

src/components/SearchBar.jsx

```
export default function SearchBar({ search, onSearchChange,
                                   category, categories, onCategoryChange }) {
  return (
    <div className="search-bar">
      <input
        placeholder="Search products..."
        value={search}
        onChange={(e) => onSearchChange(e.target.value)}
      />
      <select value={category} onChange={(e) => onCategoryChange(e.target.value)}>
        {categories.map((c) => (
          <option key={c} value={c}>
            {c}
          </option>
        ))}
      </select>
    </div>
  );
}
```

src/components/ProductCard.jsx

```
import { Link } from "react-router";

export default function ProductCard({ product }) {
  return (
    <Link to={`/${products}/${product.id}`} className="card">
      <img src={product.thumbnail} alt={product.title} />
      <h3>{product.title}</h3>
      <p className="muted">{product.category}</p>
      <p className="price">${product.price}</p>
    </Link>
  );
}
```

src/pages/ProductsPage.jsx

```

import { useState, useEffect } from "react";
import SearchBar from "../components/SearchBar.jsx";
import ProductCard from "../components/ProductCard.jsx";

const API = "https://dummyjson.com/products?limit=30";

export default function ProductsPage() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [search, setSearch] = useState("");
  const [category, setCategory] = useState("all");

  useEffect(() => {
    async function load() {
      try {
        const res = await fetch(API);
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const json = await res.json();
        setProducts(json.products);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    load();
  }, []);

  if (loading) return <p className="status">Loading products...</p>;
  if (error) return <p className="status error">Could not load: {error}</p>;

  const categories = ["all", ...new Set(products.map((p) => p.category))];

  const visible = products.filter((p) => {
    const matchesText = p.title.toLowerCase().includes(search.toLowerCase());
    const matchesCategory = category === "all" || p.category === category;
    return matchesText && matchesCategory;
  });

  return (
    <div>
      <SearchBar
        search={search}
        onSearchChange={setSearch}
        category={category}
        categories={categories}
        onCategoryChange={setCategory}
      />
      <p className="muted">{visible.length} products found</p>

      {visible.length === 0 ? (
        <p className="status">No products match your search.</p>
      ) : (
        <div className="grid">
          {visible.map((p) => (
            <ProductCard key={p.id} product={p} />
          ))}
        </div>
      )}
    </div>
  );
}

```

src/pages/ProductDetailsPage.jsx

```
import { useState, useEffect } from "react";
import { useParams, useNavigate } from "react-router";

export default function ProductDetailsPage() {
  const { id } = useParams();
  const navigate = useNavigate();
  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    async function load() {
      setLoading(true);
      setError(null);
      try {
        const res = await fetch(`https://dummyjson.com/products/${id}`, {
          signal: controller.signal,
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        setProduct(await res.json());
      } catch (err) {
        if (err.name !== "AbortError") setError(err.message);
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }

    load();
    return () => controller.abort();
  }, [id]);

  if (loading) return <p className="status">Loading product...</p>;
  if (error) return <p className="status error">Product not found ({error}).</p>;

  return (
    <div className="details">
      <button onClick={() => navigate(-1)}><- Back</button>
      <img src={product.thumbnail} alt={product.title} />
      <h2>{product.title}</h2>
      <p className="muted">
        {product.brand ?? "No brand"} · {product.category} · rating {product.rating}
      </p>
      <p>{product.description}</p>
      <p className="price">${product.price}</p>
    </div>
  );
}
```

src/index.css

```
body { margin: 0; font-family: system-ui, sans-serif; background: #f4f6f8; }
.navbar { background: #0f2a3a; padding: 14px 24px; }
.navbar a { color: #fff; font-weight: bold; text-decoration: none; }
.container { max-width: 960px; margin: 0 auto; padding: 24px; }
.search-bar { display: flex; gap: 8px; margin-bottom: 8px; }
.search-bar input { flex: 1; padding: 8px; }
.grid { display: grid; gap: 16px;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr)); }
.card { background: #fff; border-radius: 10px; padding: 12px;
  color: inherit; text-decoration: none; }
.card img { width: 100%; height: 140px; object-fit: contain; }
.card h3 { font-size: 15px; margin: 8px 0 4px; }
.muted { color: #666; font-size: 14px; }
.price { font-weight: bold; color: #087ea4; }
.status { padding: 24px; text-align: center; }
.error { color: #cf222e; }
.details img { max-width: 260px; display: block; margin: 12px 0; }
```

Key ideas to notice

- **The early returns come before the calculations.** `categories` and `visible` are only calculated once the data is ready.
- `new Set(...)` **removes duplicate categories**, and the spread turns the Set back into an array.
- `ProductCard` **is a Link**, so the whole card is clickable and uses routing.
- **The details page fetches by `id`** and lists `id` as a dependency, so moving between products loads the right data. The abort cleanup protects against slow, out-of-order responses.
- `product.brand ?? "No brand"` handles products where a field is missing, which is common with real APIs.

TIP · Extend it

Keep the search text in the URL with `useSearchParams` so it survives the Back button; add sorting by price; replace the manual fetching with your `useFetch` hook.

MINI PROJECT 3

Team Dashboard

What you will build

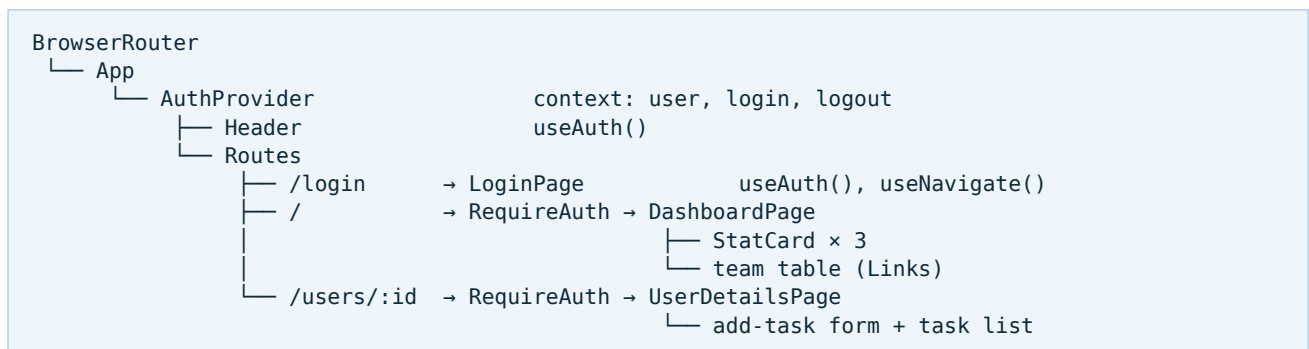
A small internal dashboard. A user logs in (a fake login, no server), sees statistics about the team's tasks, a table of team members, and can open a member's page to see their tasks and add new ones.

Concept	Where it is used
Components and props	<code>Header</code> , <code>StatCard</code> , <code>RequireAuth</code> , three pages
State	form fields, locally added tasks
Forms	the login form and the "add task" form, with validation
API calls	users and todos from JSONPlaceholder
React Router	<code>/login</code> , <code>/</code> and <code>/users/:id</code> , plus redirects
Context API	the logged-in user, available everywhere
Custom hooks	<code>useFetch</code> for data and <code>useAuth</code> for the context

API used: JSONPlaceholder.

Request	Returns
<code>https://jsonplaceholder.typicode.com/users</code>	10 users: <code>id</code> , <code>name</code> , <code>email</code> , <code>company.name</code> , ...
<code>https://jsonplaceholder.typicode.com/users/3</code>	one user
<code>https://jsonplaceholder.typicode.com/todos</code>	200 todos: <code>id</code> , <code>userId</code> , <code>title</code> , <code>completed</code>
<code>https://jsonplaceholder.typicode.com/todos?userId=3</code>	the todos of user 3

Component tree



The code

Create the folders `src/hooks`, `src/context`, `src/components` and `src/pages`. Copy `useFetch.js` from Chapter 15 into `src/hooks/`. Use the same `main.jsx` as in Mini Project 2 (with `BrowserRouter`).

`src/context/AuthContext.jsx`

```
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = (name) => setUser({ name });
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error("useAuth must be used inside <AuthProvider>");
  return context;
}
```

`src/components/RequireAuth.jsx`

```
import { Navigate } from "react-router";
import { useAuth } from "../context/AuthContext.jsx";

export default function RequireAuth({ children }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  return children;
}
```

`<Navigate>` is the component version of `useNavigate()`: when it renders, it redirects. `replace` means the protected page is not kept in the browser history, so Back does not loop.

`src/components/Header.jsx`

```
import { Link } from "react-router";
import { useAuth } from "../context/AuthContext.jsx";

export default function Header() {
  const { user, logout } = useAuth();

  return (
    <header className="header">
      <Link to="/">Team Dashboard</Link>
      {user && (
        <span>
          Hi, {user.name} <button onClick={logout}>Log out</button>
        </span>
      )}
    </header>
  );
}
```

src/components/StatCard.jsx

```
export default function StatCard({ label, value }) {
  return (
    <div className="stat">
      <p className="stat-value">{value}</p>
      <p className="stat-label">{label}</p>
    </div>
  );
}
```

src/pages/LoginPage.jsx

```
import { useState } from "react";
import { useNavigate } from "react-router";
import { useAuth } from "../context/AuthContext.jsx";

export default function LoginPage() {
  const [name, setName] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const { login } = useAuth();
  const navigate = useNavigate();

  function handleSubmit(e) {
    e.preventDefault();
    if (!name.trim() || password.length < 4) {
      setError("Enter your name and a password of at least 4 characters.");
      return;
    }
    login(name.trim());
    navigate("/");
  }

  return (
    <form className="panel" onSubmit={handleSubmit}>
      <h2>Log in</h2>
      <input placeholder="Name" value={name}
        onChange={(e) => setName(e.target.value)} />
      <input placeholder="Password" type="password" value={password}
        onChange={(e) => setPassword(e.target.value)} />
      {error && <p className="error">{error}</p>}
      <button type="submit">Log in</button>
    </form>
  );
}
```

src/pages/DashboardPage.jsx

```

import { Link } from "react-router";
import { useFetch } from "../hooks/useFetch.js";
import StatCard from "../components/StatCard.jsx";

const BASE = "https://jsonplaceholder.typicode.com";

export default function DashboardPage() {
  const users = useFetch(`${BASE}/users`);
  const todos = useFetch(`${BASE}/todos`);

  if (users.loading || todos.loading) return <p>Loading dashboard...</p>;
  if (users.error || todos.error) {
    return <p className="error">Could not load the dashboard.</p>;
  }

  const done = todos.data.filter((t) => t.completed).length;
  const percent = Math.round((done / todos.data.length) * 100);

  return (
    <div>
      <div className="stats">
        <StatCard label="Team members" value={users.data.length} />
        <StatCard label="Total tasks" value={todos.data.length} />
        <StatCard label="Completed" value={` ${percent}%`} />
      </div>

      <h2>Team</h2>
      <table className="table">
        <thead>
          <tr><th>Name</th><th>Email</th><th>Company</th></tr>
        </thead>
        <tbody>
          {users.data.map((u) => (
            <tr key={u.id}>
              <td><Link to={` /users/${u.id}`}>{u.name}</Link></td>
              <td>{u.email}</td>
              <td>{u.company.name}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

```

src/pages/UserDetailsPage.jsx

```
import { useState } from "react";
import { useParams, Link } from "react-router";
import { useFetch } from "../hooks/useFetch.js";

const BASE = "https://jsonplaceholder.typicode.com";

export default function UserDetailsPage() {
  const { id } = useParams();
  const user = useFetch(`${BASE}/users/${id}`);
  const todos = useFetch(`${BASE}/todos?userId=${id}`);
  const [newTasks, setNewTasks] = useState([]);
  const [title, setTitle] = useState("");

  function handleAdd(e) {
    e.preventDefault();
    if (!title.trim()) return;
    const task = { id: Date.now(), title: title.trim(), completed: false };
    setNewTasks((prev) => [task, ...prev]);
    setTitle("");
  }

  if (user.loading || todos.loading) return <p>Loading...</p>;
  if (user.error) return <p className="error">User not found.</p>;

  const tasks = [...newTasks, ...(todos.data ?? [])];
  const open = tasks.filter((t) => !t.completed).length;

  return (
    <div>
      <Link to="/"><- Back to dashboard</Link>
      <h2>{user.data.name}</h2>
      <p className="muted">
        {user.data.email} · {open} open of {tasks.length} tasks
      </p>

      <form onSubmit={handleAdd} className="inline-form">
        <input placeholder="New task" value={title}
          onChange={(e) => setTitle(e.target.value)} />
        <button type="submit">Add task</button>
      </form>

      <ul className="tasks">
        {tasks.map((t) => (
          <li key={t.id} className={t.completed ? "done" : ""}>
            {t.completed ? "Done" : "Open"}: {t.title}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

src/App.jsx

```
import { Routes, Route } from "react-router";
import { AuthProvider } from "../context/AuthContext.jsx";
import Header from "../components/Header.jsx";
import RequireAuth from "../components/RequireAuth.jsx";
import LoginPage from "../pages/LoginPage.jsx";
import DashboardPage from "../pages/DashboardPage.jsx";
import UserDetailsPage from "../pages/UserDetailsPage.jsx";

export default function App() {
  return (
    <AuthProvider>
      <Header />
      <main className="container">
        <Routes>
          <Route path="/login" element={<LoginPage />} />
          <Route
            path="/"
            element={<RequireAuth><DashboardPage /></RequireAuth>}
          />
          <Route
            path="/users/:id"
            element={<RequireAuth><UserDetailsPage /></RequireAuth>}
          />
          <Route path="*" element={<p>Page not found.</p>} />
        </Routes>
      </main>
    </AuthProvider>
  );
}
```

src/index.css

```
body { margin: 0; font-family: system-ui, sans-serif; background: #f4f6f8; }
.header { display: flex; justify-content: space-between; align-items: center;
  background: #0f2a3a; color: #fff; padding: 12px 24px; }
.header a { color: #fff; font-weight: bold; text-decoration: none; }
.container { max-width: 900px; margin: 0 auto; padding: 24px; }
.panel { max-width: 320px; margin: 40px auto; display: grid; gap: 10px;
  background: #fff; padding: 24px; border-radius: 10px; }
.stats { display: grid; grid-template-columns: repeat(3, 1fr); gap: 12px; }
.stat { background: #fff; border-radius: 10px; padding: 16px; }
.stat-value { font-size: 28px; font-weight: bold; margin: 0; color: #087ea4; }
.stat-label { margin: 4px 0 0; color: #666; }
.table { width: 100%; border-collapse: collapse; background: #fff; }
.table th, .table td { text-align: left; padding: 8px; border-bottom: 1px solid #eee; }
.inline-form { display: flex; gap: 8px; margin: 16px 0; }
.tasks li.done { color: #888; }
.error { color: #cf222e; }
.muted { color: #666; }
```

Key ideas to notice

- **Context plus a custom hook:** `AuthProvider` owns the user, and `useAuth()` gives every component safe access. No props are drilled through `App`.
- **RequireAuth uses composition:** it wraps a page as `children` and decides whether to show it or redirect.
- **useFetch is called twice per page**, and each call has its own independent loading and error state.
- **All hooks are called before the early returns** in `UserDetailsPage`. Hooks must run in the same order on every render, so they can never come after a conditional `return`.

- **Locally added tasks are combined with API data** during render: `[...newTasks, ...todos.data]`. They are not saved to the server, and they disappear on reload. That is expected for a practice API.

TIP · Extend it

Remember the logged-in user with `useLocalStorage`; let users tick tasks as done; add a theme context with a dark mode switch in the header.

React Cheat Sheet

Components

```
export default function Welcome() {      // capital letter, returns JSX
  return <h1>Hello</h1>;
}
import Welcome from "./Welcome.jsx";    // default import
import { Card, CardBody } from "./Card.jsx"; // named imports
<Welcome />                             // use it like a tag
```

JSX

```
<>                                     /* Fragment: one root, no extra div */
  <h1 className="title">{user.name}</h1> /* {} = any JS expression */
  <label htmlFor="email">Email</label>
  <img src={url} alt="" />             /* every tag is closed */
  <p style={{ fontSize: 14 }}>Hi</p>    /* style is an object */
</>
```

Props

```
<UserCard name="Asha" age={26} tags={["a"]} onSave={handleSave} />

function UserCard({ name, age, role = "Guest", children }) { ... } // read-only
```

Events

```
<button onClick={handleClick}>...</button> // pass, don't call
<button onClick={() => remove(item.id)}>...</button> // pass arguments
<input onChange={(e) => setText(e.target.value)} />
<form onSubmit={(e) => { e.preventDefault(); save(); }}>...</form>
```

useState

```
const [count, setCount] = useState(0);
setCount(5); // new value
setCount((prev) => prev + 1); // based on the previous value
setUser({ ...user, age: 27 }); // objects: copy, then change
setItems([...items, newItem]); // arrays: add
setItems(items.filter(i => i.id !== id)); // arrays: remove
setItems(items.map(i => (i.id === id ? { ...i, done: true } : i))); // update
```

Conditional rendering

```
if (loading) return <Spinner /> // early return
{isAdmin ? <AdminPanel /> : <UserPanel />} // either / or
{error && <p className="error">{error}</p>} // something or nothing
{items.length > 0 && <List items={items} />} // compare: never `0 &&`
```

Lists and .map()

```
<ul>
  {products.map((p) => (
    <ProductCard key={p.id} product={p} />    // stable, unique key
  ))}
</ul>
```

Forms (controlled components)

```
<input value={name} onChange={(e) => setName(e.target.value)} />
<input type="checkbox" checked={agree}
  onChange={(e) => setAgree(e.target.checked)} />
<select value={city} onChange={(e) => setCity(e.target.value)}>...</select>

// one handler for many fields (each input has a matching name="...")
setForm((prev) => ({ ...prev, [e.target.name]: e.target.value }));
```

useEffect

```
useEffect(() => { ... });           // after every render
useEffect(() => { ... }, []);       // once, after the first render
useEffect(() => { ... }, [id]);     // first render + whenever id changes

useEffect(() => {
  const timer = setInterval(tick, 1000);
  return () => clearInterval(timer); // cleanup
}, []);
```

Fetch

```
useEffect(() => {
  async function load() {
    try {
      const res = await fetch(url);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setData(await res.json());
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }
  load();
}, [url]);
```

React Router

```
import { BrowserRouter, Routes, Route, Link, NavLink,
  Navigate, useNavigate, useParams } from "react-router";

<BrowserRouter><App /></BrowserRouter>           // once, in main.jsx
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/products/:id" element={<Details />} />
  <Route path="*" element={<NotFound />} />
</Routes>
<Link to="/products/3">View</Link>
const { id } = useParams();                     // always a string
const navigate = useNavigate(); navigate("/");   // navigate(-1) = back
```

Context

```
const ThemeContext = createContext(null);           // 1. create
<ThemeContext.Provider value={{ theme, toggle }}>   // 2. provide
  {children}
</ThemeContext.Provider>
const { theme, toggle } = useContext(ThemeContext); // 3. consume
```

Custom hooks

```
function useToggle(initial = false) {              // name starts with "use"
  const [on, setOn] = useState(initial);            // may call other hooks
  return [on, () => setOn((v) => !v)];              // returns data/functions, not JSX
}
const [open, toggleOpen] = useToggle();            // each call has its own state
```

REMEMBER · The Rules of Hooks

Call hooks only at the **top level** of a component or custom hook: never inside **if**, loops, nested functions or after an early **return**. Call them only from React components or other hooks.

How the Concepts Fit Together

The concept map

Each concept builds on the ones before it:

Component	a function that returns UI
↓	
JSX	the UI it returns, with {expressions}
↓	
Props	data passed down from a parent
↓	
Events	the user does something → a handler runs
↓	
State	handlers update state → React re-renders
↓	
Conditional Rendering	state decides WHAT to show (loading? error? empty?)
↓	
Lists	arrays in state become lists of components (.map + key)
↓	
Forms	inputs controlled by state; submit sends the data
↓	
useEffect	after rendering, sync with the outside world
↓	
API	an effect fetches data → it is stored in state
↓	
Router	the URL decides which page component is shown
↓	
Context	data many components need, without prop drilling
↓	
Custom Hooks	repeated stateful logic, packaged for reuse

One click, all concepts: a walkthrough

Here is what happens in the **Product Explorer** (Mini Project 2) from the moment the user opens it until they view a product. Almost every concept from this workbook plays a part.

Step	What happens	Concepts
1	The browser loads <code>index.html</code> . <code>main.jsx</code> renders <code><App /></code> inside <code>BrowserRouter</code> .	Components, Router
2	The URL is <code>/</code> , so <code>Routes</code> renders <code>ProductsPage</code> .	Router
3	<code>ProductsPage</code> runs: <code>loading</code> is <code>true</code> , so it returns "Loading products...".	State, Conditional rendering, JSX
4	After the screen is painted, the effect runs and calls <code>fetch()</code> .	useEffect, API
5	The JSON arrives. <code>setProducts</code> and <code>setLoading(false)</code> trigger a re-render.	State, API
6	The products array is filtered by the search text, then <code>.map()</code> creates a <code>ProductCard</code> for each item, passing <code>product</code> as a prop.	Lists, Props, Composition

Step	What happens	Concepts
7	The user types "phone". <code>onChange</code> calls <code>setSearch</code> ; the component re-renders and the filtered list shrinks. No new request is needed.	Events, Forms, State
8	The user clicks a card. <code>Link</code> changes the URL to <code>/products/7</code> without reloading.	Events, Router
9	<code>ProductDetailsPage</code> reads <code>id</code> with <code>useParams()</code> and fetches that product in an effect that depends on <code>id</code> .	Router, <code>useEffect</code> , API
10	If this page used a shared cart, it would read it with <code>useContext</code> , and <code>useFetch</code> could replace the manual fetching code.	Context, Custom hooks

The mental model to keep

REMEMBER • $UI = f(state)$

A React component is a function from **data** to **UI**. You never change the screen directly. You change **state**, and React calls your functions again to work out what the screen should look like. Props carry data down, callbacks carry events up, effects synchronise with the outside world, and context and hooks help you share data and logic as the app grows.

Final Practice Challenge: Recipe Finder

Build this application **on your own**. The solution is in the next chapter, but try to finish (or at least get far) before you look at it.

Requirements

Build a **Recipe Finder** that loads recipes from a public API. Users can search and filter recipes, view a recipe's details, save favourites, and add their own simple recipes to a personal list.

UI requirements

- A **header** on every page with the app name and links to **Recipes**, **Favourites (N)** and **My Recipes**, where N is the number of favourites.
- **Recipes page** (/): a search input, a difficulty dropdown (All, Easy, Medium, Hard) and a grid of recipe cards. Each card shows the image, name, cuisine, difficulty and total time (prep + cook).
- **Recipe details page** (/recipes/:id): the image, name, cuisine, servings, ingredients as a bulleted list, instructions as a numbered list, a "Save to favourites" / "Remove from favourites" button, and a Back button.
- **Favourites page** (/favorites): the saved recipes as cards, or the message "No favourites yet" when there are none.
- **My Recipes page** (/my-recipes): a form (name, cuisine, total minutes, difficulty) and, below it, the list of recipes the user added.
- A simple **404** page for unknown URLs.

Data requirements

Request	Returns
https://dummyjson.com/recipes?limit=30	{ recipes: [...], total, skip, limit }
https://dummyjson.com/recipes/1	one recipe object

Useful recipe fields: **id**, **name**, **ingredients** (array of strings), **instructions** (array of strings), **prepTimeMinutes**, **cookTimeMinutes**, **servings**, **difficulty** ("Easy", "Medium" or "Hard"), **cuisine**, **image**, **rating**.

Functional requirements

1. Show a loading message while data loads, and an error message if a request fails.
2. The search filters recipes by name, ignoring upper and lower case. The dropdown filters by difficulty. Both work together.
3. Show "No recipes match your search" when the filters remove every recipe.
4. Favourites are shared between the details page, the favourites page and the header count.
5. The My Recipes form validates: the name is required, the minutes must be a number greater than 0. Show errors under the form. On success, add the recipe to the list and clear the form.

6. The favourites and the user's own recipes may be lost on page reload (keeping them is a bonus task).

React concepts you must use

Concept	Minimum use
Components and props	at least 6 components, including a reusable <code>RecipeCard</code>
State and events	search text, difficulty filter, form fields
Conditional rendering	loading, error, empty and "is it a favourite?" states
Lists and keys	the recipe grid, ingredients and instructions
Forms	the controlled My Recipes form with validation
useEffect / API	through a <code>useFetch</code> custom hook
React Router	4 routes plus 404, <code>Link</code> , <code>useParams</code> , <code>useNavigate</code>
Context API	one context for favourites and the user's own recipes
Custom hooks	<code>useFetch</code> and a <code>useRecipeBook()</code> hook for the context

Hints

- Plan first: sketch the four pages and draw the component tree. Decide which state is **global** (context) and which is **local** to a page.
- Store favourites as an array of small objects, for example `{ id, name, image, cuisine, difficulty, time }`, so the favourites page does not need to fetch anything.
- `isFavorite(id)` can be written as `favorites.some((r) => r.id === id)`.
- `useParams()` gives you a string. The API accepts it directly in the URL, but convert it with `Number(id)` before comparing it with a recipe's `id`.
- Reuse `RecipeCard` on both the Recipes page and the Favourites page.
- For the form, `Number(form.minutes)` turns the input's string into a number; `Number.isNaN()` helps with validation.
- Build in this order: routes and empty pages → the recipe list with `useFetch` → search and filter → details page → context and favourites → the form.

STOP AND TRY FIRST

The solution starts on the next page. Give yourself at least one serious attempt first. Getting stuck and then un-stuck is where most of the learning happens.

Final Challenge: Solution

Folder structure

```
src/
├── main.jsx           BrowserRouter + RecipeBookProvider
├── App.jsx           Header + Routes
├── index.css
├── hooks/
│   └── useFetch.js    copied from Chapter 15
├── context/
│   └── RecipeBookContext.jsx    favourites + my recipes + useRecipeBook()
├── components/
│   ├── Header.jsx
│   └── RecipeCard.jsx
└── pages/
    ├── RecipesPage.jsx
    ├── RecipeDetailsPage.jsx
    ├── FavoritesPage.jsx
    └── MyRecipesPage.jsx
```

The code

src/context/RecipeBookContext.jsx

```
import { createContext, useContext, useState } from "react";

const RecipeBookContext = createContext(null);

export function RecipeBookProvider({ children }) {
  const [favorites, setFavorites] = useState([]);
  const [myRecipes, setMyRecipes] = useState([]);

  const isFavorite = (id) => favorites.some((r) => r.id === id);

  function toggleFavorite(recipe) {
    setFavorites((prev) =>
      prev.some((r) => r.id === recipe.id)
        ? prev.filter((r) => r.id !== recipe.id)
        : [...prev, recipe]
    );
  }

  function addMyRecipe(recipe) {
    setMyRecipes((prev) => [{ ...recipe, id: Date.now() }, ...prev]);
  }

  const value = { favorites, isFavorite, toggleFavorite, myRecipes, addMyRecipe };

  return (
    <RecipeBookContext.Provider value={value}>
      {children}
    </RecipeBookContext.Provider>
  );
}

export function useRecipeBook() {
  const context = useContext(RecipeBookContext);
  if (!context) {
    throw new Error("useRecipeBook must be used inside <RecipeBookProvider>");
  }
  return context;
}
```

src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router";
import { RecipeBookProvider } from "../context/RecipeBookContext.jsx";
import App from "../App.jsx";
import "../index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <RecipeBookProvider>
        <App />
      </RecipeBookProvider>
    </BrowserRouter>
  </StrictMode>
);
```

src/App.jsx

```
import { Routes, Route, Link } from "react-router";
import Header from "../components/Header.jsx";
import RecipesPage from "../pages/RecipesPage.jsx";
import RecipeDetailsPage from "../pages/RecipeDetailsPage.jsx";
import FavoritesPage from "../pages/FavoritesPage.jsx";
import MyRecipesPage from "../pages/MyRecipesPage.jsx";

function NotFound() {
  return (
    <div className="status">
      <h2>Page not found</h2>
      <Link to="/">Back to recipes</Link>
    </div>
  );
}

export default function App() {
  return (
    <>
      <Header />
      <main className="container">
        <Routes>
          <Route path="/" element={<RecipesPage />} />
          <Route path="/recipes/:id" element={<RecipeDetailsPage />} />
          <Route path="/favorites" element={<FavoritesPage />} />
          <Route path="/my-recipes" element={<MyRecipesPage />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>
    </>
  );
}
```

src/components/Header.jsx

```
import { NavLink } from "react-router";
import { useRecipeBook } from "../context/RecipeBookContext.jsx";

const linkClass = ({ isActive }) => (isActive ? "active" : "");

export default function Header() {
  const { favorites } = useRecipeBook();

  return (
    <header className="header">
      <strong>Recipe Finder</strong>
      <nav>
        <NavLink to="/" end className={linkClass}>Recipes</NavLink>
        <NavLink to="/favorites" className={linkClass}>
          Favourites ({favorites.length})
        </NavLink>
        <NavLink to="/my-recipes" className={linkClass}>My Recipes</NavLink>
      </nav>
    </header>
  );
}
```

src/components/RecipeCard.jsx

```
import { Link } from "react-router";

export default function RecipeCard({ recipe }) {
  return (
    <Link to={`/recipes/${recipe.id}`} className="card">
      <img src={recipe.image} alt={recipe.name} />
      <h3>{recipe.name}</h3>
      <p className="muted">
        {recipe.cuisine} · {recipe.difficulty} · {recipe.time} min
      </p>
    </Link>
  );
}
```

RecipeCard expects a small "summary" object with a **time** field. Both the recipes page and the favourites use the same shape, so one card works everywhere.

src/pages/RecipesPage.jsx

```

import { useState } from "react";
import { useFetch } from "../hooks/useFetch.js";
import RecipeCard from "../components/RecipeCard.jsx";

const API = "https://dummyjson.com/recipes?limit=30";
const LEVELS = ["All", "Easy", "Medium", "Hard"];

export function toSummary(r) {
  return {
    id: r.id,
    name: r.name,
    image: r.image,
    cuisine: r.cuisine,
    difficulty: r.difficulty,
    time: r.prepTimeMinutes + r.cookTimeMinutes,
  };
}

export default function RecipesPage() {
  const { data, loading, error } = useFetch(API);
  const [search, setSearch] = useState("");
  const [level, setLevel] = useState("All");

  if (loading) return <p className="status">Loading recipes...</p>;
  if (error) return <p className="status error">Could not load: {error}</p>;

  const visible = data.recipes
    .filter((r) => r.name.toLowerCase().includes(search.toLowerCase()))
    .filter((r) => level === "All" || r.difficulty === level)
    .map(toSummary);

  return (
    <div>
      <div className="filters">
        <input
          placeholder="Search recipes..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
        />
        <select value={level} onChange={(e) => setLevel(e.target.value)}>
          {LEVELS.map((l) => (
            <option key={l}>{l}</option>
          ))}
        </select>
      </div>

      {visible.length === 0 ? (
        <p className="status">No recipes match your search.</p>
      ) : (
        <div className="grid">
          {visible.map((r) => (
            <RecipeCard key={r.id} recipe={r} />
          ))}
        </div>
      )}
    </div>
  );
}

```

src/pages/RecipeDetailsPage.jsx

```

import { useParams, useNavigate } from "react-router";
import { useFetch } from "../hooks/useFetch.js";
import { useRecipeBook } from "../context/RecipeBookContext.jsx";
import { toSummary } from "../RecipesPage.jsx";

export default function RecipeDetailsPage() {
  const { id } = useParams();
  const navigate = useNavigate();
  const { data: recipe, loading, error } = useFetch(
    `https://dummyjson.com/recipes/${id}`
  );
  const { isFavorite, toggleFavorite } = useRecipeBook();

  if (loading) return <p className="status">Loading recipe...</p>;
  if (error) return <p className="status error">Recipe not found.</p>;

  const saved = isFavorite(recipe.id);

  return (
    <article className="details">
      <button onClick={() => navigate(-1)}><- Back</button>
      <img src={recipe.image} alt={recipe.name} />
      <h2>{recipe.name}</h2>
      <p className="muted">
        {recipe.cuisine} · serves {recipe.servings} · rating {recipe.rating}
      </p>

      <button onClick={() => toggleFavorite(toSummary(recipe))}>
        {saved ? "Remove from favourites" : "Save to favourites"}
      </button>

      <h3>Ingredients</h3>
      <ul>
        {recipe.ingredients.map((item) => (
          <li key={item}>{item}</li>
        ))}
      </ul>

      <h3>Instructions</h3>
      <ol>
        {recipe.instructions.map((step, index) => (
          <li key={index}>{step}</li>
        ))}
      </ol>
    </article>
  );
}

```

Using `index` as the key for the instructions is acceptable here: the list comes from the server, and we never reorder, filter or delete steps.

src/pages/FavoritesPage.jsx

```
import { Link } from "react-router";
import { useRecipeBook } from "../context/RecipeBookContext.jsx";
import RecipeCard from "../components/RecipeCard.jsx";

export default function FavoritesPage() {
  const { favorites } = useRecipeBook();

  if (favorites.length === 0) {
    return (
      <div className="status">
        <p>No favourites yet.</p>
        <Link to="/">Find a recipe</Link>
      </div>
    );
  }

  return (
    <div className="grid">
      {favorites.map((r) => (
        <RecipeCard key={r.id} recipe={r} />
      ))}
    </div>
  );
}
```

src/pages/MyRecipesPage.jsx

```

import { useState } from "react";
import { useRecipeBook } from "../context/RecipeBookContext.jsx";

const emptyForm = { name: "", cuisine: "", minutes: "", difficulty: "Easy" };

export default function MyRecipesPage() {
  const { myRecipes, addMyRecipe } = useRecipeBook();
  const [form, setForm] = useState(emptyForm);
  const [errors, setErrors] = useState([]);

  function handleChange(e) {
    const { name, value } = e.target;
    setForm((prev) => ({ ...prev, [name]: value }));
  }

  function handleSubmit(e) {
    e.preventDefault();
    const found = [];
    const minutes = Number(form.minutes);
    if (!form.name.trim()) found.push("Name is required.");
    if (Number.isNaN(minutes) || minutes <= 0) {
      found.push("Minutes must be a number greater than 0.");
    }
    setErrors(found);
    if (found.length > 0) return;

    addMyRecipe({ ...form, name: form.name.trim(), minutes });
    setForm(emptyForm);
  }

  return (
    <div>
      <form className="panel" onSubmit={handleSubmit}>
        <h2>Add my recipe</h2>
        <input name="name" placeholder="Recipe name"
          value={form.name} onChange={handleChange} />
        <input name="cuisine" placeholder="Cuisine (optional)"
          value={form.cuisine} onChange={handleChange} />
        <input name="minutes" placeholder="Total minutes"
          value={form.minutes} onChange={handleChange} />
        <select name="difficulty" value={form.difficulty} onChange={handleChange}>
          <option>Easy</option>
          <option>Medium</option>
          <option>Hard</option>
        </select>
        {errors.map((err) => (
          <p key={err} className="error">{err}</p>
        ))}
        <button type="submit">Add recipe</button>
      </form>

      <h2>My recipes ({myRecipes.length})</h2>
      {myRecipes.length === 0 && <p className="muted">You have not added any yet.</p>}
      <ul>
        {myRecipes.map((r) => (
          <li key={r.id}>
            <strong>{r.name}</strong>
            {r.cuisine && ` · ${r.cuisine}`} · {r.difficulty} · {r.minutes} min
          </li>
        ))}
      </ul>
    </div>
  );
}

```

src/index.css

```
body { margin: 0; font-family: system-ui, sans-serif; background: #faf7f2; }
.header { display: flex; justify-content: space-between; align-items: center;
  flex-wrap: wrap; gap: 8px; padding: 14px 24px;
  background: #3d2c1e; color: #fff; }
.header nav { display: flex; gap: 16px; }
.header a { color: #f3e2c7; text-decoration: none; }
.header a.active { color: #fff; font-weight: bold; }
.container { max-width: 960px; margin: 0 auto; padding: 24px; }
.filters { display: flex; gap: 8px; margin-bottom: 16px; }
.filters input { flex: 1; padding: 8px; }
.grid { display: grid; gap: 16px;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); }
.card { background: #fff; border-radius: 12px; overflow: hidden;
  color: inherit; text-decoration: none; padding-bottom: 8px; }
.card img { width: 100%; height: 150px; object-fit: cover; }
.card h3 { font-size: 16px; margin: 8px 12px 4px; }
.card p { margin: 0 12px; }
.muted { color: #6b6b6b; font-size: 14px; }
.status { text-align: center; padding: 32px; }
.error { color: #b42318; margin: 0; }
.panel { display: grid; gap: 10px; max-width: 380px; background: #fff;
  padding: 20px; border-radius: 12px; }
.details img { width: 100%; max-width: 420px; border-radius: 12px; margin: 12px 0; }
```

Why the solution is built this way

- **Global vs local state.** Favourites and "my recipes" are needed by several pages and the header, so they live in context. The search text, the filter and the form fields are only needed on one page, so they are ordinary local state.
- `useRecipeBook()` hides `useContext` and gives a clear error if the provider is missing.
- `toSummary()` turns a full API recipe into the small object that `RecipeCard` and the favourites need. Keeping one shape means one card component works on every page.
- **`useFetch` everywhere.** Neither page contains any fetching code of its own; they only decide what to show for loading, error and data.
- **The details page depends on the URL.** Because `useFetch` re-runs when its URL changes, opening a different recipe always loads the right one.

TIP • Bonus tasks

1. Keep favourites and "my recipes" after a reload with the `useLocalStorage` hook from Chapter 15.
2. Add a "Meal type" filter using the `mealType` array.
3. Let users delete their own recipes.
4. Add a dark mode with a second context.

Quick Check Answers

Chapter 1: Component Basics

1. A JavaScript function that returns a piece of UI (JSX).
2. React treats lowercase tags as built-in HTML elements; a capital letter tells it to call your component.
3. It makes the function (or value) available for other files to import.
4. React. You write `<Welcome />` and React calls the function when it renders.

Chapter 2: JSX

1. "Evaluate this JavaScript expression and insert the result."
2. `class` is a reserved word in JavaScript.
3. `<> </>`, a wrapper that groups elements without adding a real element to the page. Use it when you need one root but do not want an extra `<div>`.
4. No. `if` is a statement, and braces only accept expressions. Use a ternary, `&&`, or an `if` above the `return`.
5. An error: "Objects are not valid as a React child". Render a property such as `{user.name}` instead.

Chapter 3: Props

1. One way: from parent to child.
2. `"26"` is a string; `{26}` is a number.
3. No. Props are read-only.
4. With a default in the destructuring: `function Card({ role = "Guest" })`.
5. Nothing for that part, because `title` is `undefined`.

Chapter 4: Events

1. `onClick={save}` passes the function to be called on click; `onClick={save()}` calls it immediately during render.
2. Wrap it in an arrow function: `onClick={() => remove(id)}`.
3. It stops the browser's default form behaviour, which would reload the page.
4. The parent passes a function as a prop (for example `onLike`), and the child calls it.

Chapter 5: State

1. React does not know it changed, so it does not re-render, and the variable resets to its initial value every time the function runs.
2. The current value and a setter function.
3. When the new value depends on the previous value, especially if you update several times in a row.
4. It mutates the existing array, so React does not see a change. Write `setItems([...items, "x"])`.
5. No. `count` keeps its old value until the next render.

Chapter 6: Props vs State

1. Props are owned by the parent; state is owned by the component that creates it.
2. No.
3. Moving state up to the closest common parent of the components that need it, then passing it down as props.
4. Each `ProductCard` calls `useState` itself, so each has its own separate state.
5. Neither: calculate it during render.

Chapter 7: Conditional Rendering

1. An `if` with an early `return`, the ternary operator `? :`, and the `&&` operator (also: `return null`).
2. When the alternative is "show nothing".
3. When `count` is `0`, `0 && ...` evaluates to `0`, and React renders numbers.
4. Nothing.

Chapter 8: Lists

1. An array of JSX elements, which React renders one after another.
2. It gives each item a stable identity so React can match items between renders.
3. After a deletion, the positions shift, so keys point at different items, and React may reuse the wrong elements and their state.
4. On the outermost element returned by the `.map()` callback.
5. Use `.filter()` before `.map()`.

Chapter 9: Composition

1. The special prop that contains whatever you place between a component's opening and closing tags.
2. When the component is a wrapper and the content inside varies a lot (cards, layouts, modals).
3. With braces: `import { Card } from './Card.jsx'`.
4. You are copying the same JSX again; the part has its own clear job; the file is getting very long.

Chapter 10: Forms

1. Its value comes from state (`value` or `checked`) and it updates that state in `onChange`.
2. `onSubmit` also fires when the user presses Enter, and it lets you call `preventDefault()` on the form submission.
3. With `checked={state}` and `onChange` reading `e.target.checked`.
4. It creates a property whose name is the value of the variable `name`.
5. Set the state back to its initial values.

Chapter 11: `useEffect`

1. Work that affects something outside React's rendering, such as fetching data, timers, changing `document.title` or using `localStorage`.
2. After every render; once after the first render; after the first render and whenever `count` changes.
3. Before the effect runs again, and when the component is removed from the page.
4. StrictMode in development mounts components twice to help you find missing cleanups.
5. No. Calculate it directly during render.

Chapter 12: API Calls

1. The data, a loading flag and an error.
2. An async function returns a Promise, but React expects an effect to return nothing or a cleanup function.
3. No. It only throws on network failures; you must check `response.ok`.
4. It fetches again whenever `userId` changes.
5. It cancels an out-of-date request so that a slow old response cannot overwrite newer data (a race condition).

Chapter 13: React Router

1. So that each screen has its own URL, which allows bookmarks, shared links and the Back button, without full page reloads.
2. `<Link>` changes the URL inside the app without reloading; `<a href>` reloads the whole page and loses state.
3. With `const { id } = useParams();`. It is always a string.
4. Any URL not matched by another route, which makes it useful for a 404 page.
5. When you navigate from code, for example after a successful form submit or login.

Chapter 14: Context API

1. Passing props through components that do not use them, just to reach a deeper component.
2. `createContext`, wrap the tree in a Provider with a `value`, and read it with `useContext`.
3. The default value given to `createContext`.
4. Every component that reads that context with `useContext`.
5. When the data is only needed one or two levels down (or it changes very often and is used widely).

Chapter 15: Custom Hooks

1. Its name starts with `use` and it calls other hooks.
2. No. Each call has its own state; they fetch and store the data separately.
3. A custom hook returns data and functions; a component returns JSX.
4. So React and the linter can recognise it and enforce the Rules of Hooks.